

UD4 Documentación de aplicaciones



Módulo: Desarrollo de interfaces

Contenido

1. Documentación	3
2. Documentación de proyectos	3
2.1. Manual de instalación	3
2.2. Documentación de configuración	4
2.3. Manual de usuario	4
2.4. Guía de usuario	5
2.5. Guía rápida.....	5
2.6. Manual de administración.....	5
3. Técnicas de documentación en código.....	5
3.1 Introducción: tendencias.	5
Para un proyecto con Visual Studio 2022 y C#	6
3.2. Generar documentación XML	6
3.3. Herramienta GhostDoc.....	7
3.3.1. Configuración de GhostDoc.	8
3.3.2. Generar documentación con GhostDoc.....	12
3.4. Herramienta SandCastle.	13
3.4.1. Instalación de SandCastle Help File Builder.	13
3.5 DocFX	28
4. Herramienta HelpNDoc.	28
NO lee tu código fuente C#.....	29
4.1. Generar documentación con HelpNDoc.	29
4.2. Abrir documentación desde Visual Studio.	38
5. Incorporación de ayuda en Visual Studio.....	39
5.1. Ejemplo de uso del control HelpProvider.	40
5. 2. Ayuda contextual.....	42
5.2.1. ToolTips.....	42
5.2.1.1. Introducción al componente ToolTip de Windows Forms.....	43
5.2.1.2. Establecer ToolTips para controles en Formularios de Windows.	43

1. Documentación

La creación de documentos suele ser una tarea postergada a la finalización del proyecto y se la considera más bien como una carga de trabajo en lugar de valorar esta tarea con la importancia real que tiene.

La documentación es un elemento esencial en cualquier tipo de proyecto y en especial en los proyectos de tecnologías de la información donde la reutilización de código y el trabajar a partir de soluciones de terceros es algo habitual. Sin documentación la tarea de trabajar a partir de productos de terceros se hace casi imposible. Además, el diseño o creación de documentación de ayuda insuficiente redundaría en el concepto que el usuario tiene de la aplicación.

La tarea de creación de la ayuda por parte de los desarrolladores se vuelve una tarea muy tediosa y dedican poca atención como norma general. De ahí la necesidad de encontrar algún mecanismo que permita facilitar y automatizar la generación de documentación de ayuda.

2. Documentación de proyectos

Durante el desarrollo de un proyecto se generan ciertos tipos de documentación que se entregan a los usuarios correspondientes de forma que estos tengan la información funcional, técnica y de operación sobre el producto desarrollado.

2.1. Manual de instalación

Contiene todos los detalles necesarios para llevar a cabo la instalación de la aplicación en diferentes entornos. Se contemplan en esta documentación aspectos referidos al software como pueden ser diferentes sistemas operativos, otras aplicaciones instaladas, dependencias de otros programas previamente instalados que si no están presentes se deben instalar (a mano o a través del propio instalador de la aplicación) y aspectos referidos a características hardware mínimas exigidas (memoria RAM, disco duro, tarjeta gráfica, etc.)

Se explican también distintas posibilidades de instalación como pueden ser el cambio de directorios por defecto o la generación o no de, por ejemplo, archivos de log, así como la ubicación de los archivos de configuración, tipo de instalación (completa, estándar, mínima) o diferentes aspectos que pueden modificar la apariencia (skins), los menús que se muestran o diferentes ajustes de funcionamiento según las necesidades (seguridad, rapidez, control de procesos, mejor presentación, etc.)

De esta documentación suele haber dos versiones, una reducida para instalación rápida con valores por defecto, sin profundizar y otra detallada donde se explican ampliamente cada uno de los aspectos anteriormente nombrados.

2.2. Documentación de configuración

Es una parte de la documentación dedicada a explicar todos los parámetros de configuración de la aplicación, así como el efecto que producen cada uno de los cambios en el funcionamiento de dicha aplicación.

En esta documentación se mezclan aspectos software, hardware, visuales, etc. Por ejemplo, en el IDE de NetBeans la configuración permite modificar la visualización del texto o de los elementos visuales dentro del IDE, permite modificar colores, así como el tipo de letra. Asimismo, permite definir cómo se ejecutarán las aplicaciones o las librerías por defecto que se usarán o los directorios de destino del resultado de nuestros proyectos. En otras aplicaciones, por ejemplo, como máquinas virtuales se permite definir las características hardware (disco duro, memoria RAM, etc.) de la máquina que queremos simular.

En el documento referido a la configuración se debe especificar cómo realizar estos cambios en la configuración de la aplicación, así como qué consecuencias tendrán dichos cambios en el funcionamiento y presentación de la aplicación.

Es habitual que se encuentren varios niveles de configuración. Lo más habitual son dos niveles: Configuración (normal) y Configuración Avanzada. Dependiendo de lo compleja que sea la aplicación puede ocurrir que además de la guía de configuración (normal) se genere una guía de configuración avanzada en la que se detallan los aspectos y labores más complejos de la administración de la aplicación que requieren de mayor experiencia y conocimiento por parte de los usuarios (administradores).

2.3. Manual de usuario

Hay dos versiones: la rápida y la detallada. La diferencia entre una y otra es el detalle con el que explican las posibilidades de la aplicación. En muchos casos se describen las funciones que puede realizar el usuario final desde el punto de vista estrictamente práctico y funcional sin preocuparnos de aspectos técnicos (que suponen salvados o tenidos en cuenta por los administradores para lo que se han ayudado de las guías de instalación y funcionamiento).

En muchos casos se hace uso en esta documentación de la interfaz de usuario de la aplicación para completar la explicación. Se explican al usuario los pasos que debe seguir para realizar su trabajo a través de las ventanas/pantallas/páginas del interfaz de usuario con lo que el usuario final será capaz de identificar dichas imágenes en el interfaz real de la aplicación.

En este tipo de documentación hay que tener en cuenta que el nivel de los usuarios que usan la aplicación puede no ser homogéneo y en algunos casos será necesario realizar varias versiones: principiante, inicial, medio, avanzado, experto. De forma que en cada caso se profundiza más en las explicaciones. Así para niveles iniciales se explicará el funcionamiento de la aplicación, pero desde el punto de vista puramente funcional (describiendo cada acción y los valores que debemos proporcionar o elegir para llevarlas a cabo) y además se elegirán acciones fáciles de realizar. Según se va subiendo

de nivel, experiencia y capacidad se describen acciones más complejas que requieren una mayor y más amplia explicación.

Una estructura frecuente de los manuales de usuario incluye una introducción al producto en cuestión, un índice con los contenidos del manual, la guía en sí misma, una sección de problemas frecuentes y su forma de solucionarlos, los datos de contacto y un glosario.

2.4. Guía de usuario

La guía de usuario tiene un objetivo idéntico al manual de usuario, diferenciándose de este en la extensión y cantidad de la información facilitada. Las guías de usuario tienen como objetivo indicar de forma rápida y sencilla ciertas funcionalidades del producto de forma que el cliente puede operar de forma inmediata.

2.5. Guía rápida

Documentación complementaria a la documentación de usuario o administrador y recoge referencias a las guías comentadas. En su contenido se detalla de forma breve y muy sintetizada la forma de realizar la tarea.

2.6. Manual de administración

Permite generar la guía para conocer los detalles de la instalación, la administración del sistema la configuración de los diferentes componentes de la aplicación o sistema. Este tipo de manuales son utilizados por los administradores de TI, ya que disponen de los máximos privilegios a nivel operativo en materia de TI dentro de la organización.

3. Técnicas de documentación en código

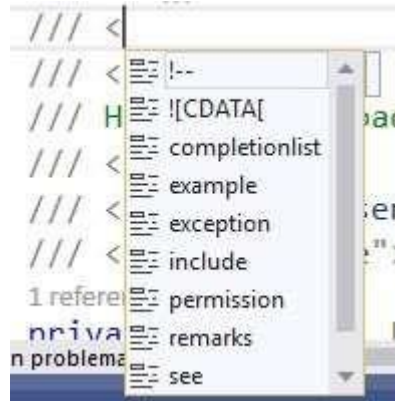
3.1 Introducción: tendencias.

En el código es posible crear documentación de ayuda mediante el uso de metainformación a modo de comentarios en el código. Esta metainformación se crea mediante el uso de caracteres `///` (tres barras), escritas delante de cualquier método o propiedad, se genera automáticamente el siguiente bloque de código:

```
/// <summary>
/// Handles the Load event of the Form1 control.
/// </summary>
/// <param name="sender">The source of the event.</param>
/// <param name="e">The <see cref="EventArgs"/> instance containing the event data.</param>
1 referencia
private void Form1_Load(object sender, EventArgs e)
```

Dependiendo del elemento a documentar y de su estructura se crean por defecto unos u otros campos como queda de manifiesto en la figura adjunta.

En cualquier caso, es posible utilizar campos adicionales, acotados entre símbolos `<` y `>`, según la lista desplegable:



🔊 Para un proyecto con Visual Studio 2022 y C#

- Para ESCRIBIR la documentación (Comentarios XML) podemos optar por estos dos enfoques:
 1. (Se dispone de GitHub Copilot): No instales GhostDoc. Usa Copilot. Simplemente escribe `///` y deja que la IA redacte la descripción; será mucho más precisa y entenderá tu lógica de negocio, tus clases con sus métodos, etc.
 2. (No disponemos de IA integrada): Instalaremos GhostDoc Community (Gratis). Nos ahorrará tiempo escribiendo la estructura básica y te corregirá la ortografía, aunque luego tendrás que editar el texto manualmente para que sea explicativo.
- Para GENERAR el archivo final (Web/PDF/CHM):
 1. Podemos usar DocFX (para convertir los comentarios de código en una web).
 2. Y también usar HelpNDoc (para redactar el manual de usuario visual para los formularios de la vista).



GhostDoc ya no es "obligatorio" como antes. Si tienes acceso a herramientas de IA en Visual Studio, estas son la alternativa moderna superior para redactar. GhostDoc queda como una buena herramienta de utilidad (spell check, estructura) si prefieres no usar IA.

3.2. Generar documentación XML

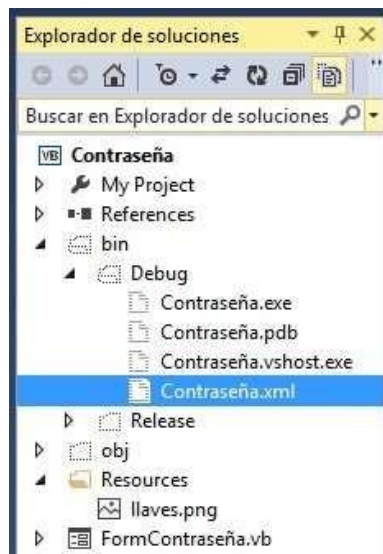
Para generar la documentación en XML dentro de las propiedades del proyecto debemos acceder a la opción Compilación y marcar la opción Archivo de documentación XML

La propiedad Archivo de documentación XML determina si un archivo XML se generará durante la compilación. Cuando esta opción está seleccionada, la documentación XML se emite automáticamente en un archivo XML que tendrá el mismo nombre que el proyecto y la extensión .XML.

Este fichero se genera al compilar el código del proyecto. De forma predeterminada, el archivo se crea bajo la ruta de acceso de resultados especificada, por ejemplo, "bin\Debug\ProjectName.XML".

Si queremos que ese archivo se vuelva a generar debemos volver a generar el proyecto pues el archivo XML se genera en la compilación.

Por ejemplo, aquí vemos el fichero Contraseña.xml en el Explorador de soluciones.



Su contenido es el siguiente:

```
<?xml version="1.0"?>
<doc>
<assembly>
<name>
Contraseña
</name>
</assembly>
<members>
<member name="P:Contraseña.My.Resources.Resources.ResourceManager">
<summary>
Devuelve la instancia de ResourceManager almacenada en caché utilizada por esta clase.
</summary>
</member><member name="P:Contraseña.My.Resources.Resources.Culture">
<summary>
Reemplaza la propiedad CurrentUICulture del subproceso actual para todas las
búsquedas de recursos mediante esta clase de recurso con establecimiento inflexible de tipos.
</summary>
</member><member name="T:Contraseña.My.Resources.Resources">
<summary>
Clase de recurso con establecimiento inflexible de tipos, para buscar cadenas traducidas, etc.
</summary>
</member>
</members>
</doc>
```

El archivo de documentación es un archivo XML que reúne todos los comentarios que hemos hecho en nuestros métodos, propiedades, etc.

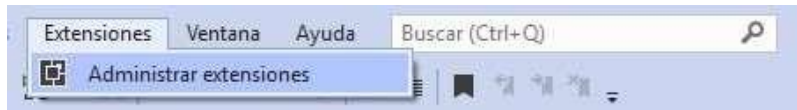
En general este archivo XML puede ser trabajado con otras herramientas para generar un archivo CHM o de ayuda.

A continuación, vamos a ver algunas de esas herramientas.

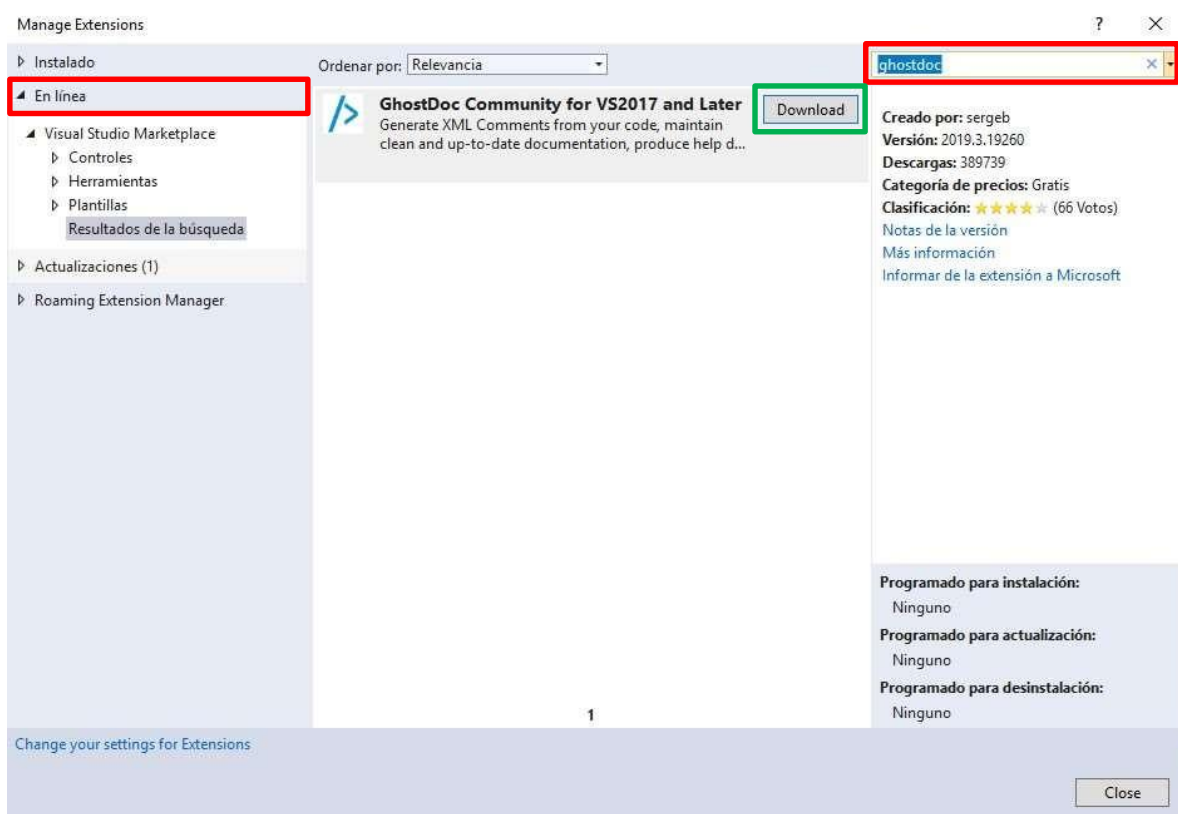
3.3. Herramienta GhostDoc.

GhostDoc es una extensión para Visual Studio que genera comentarios de documentación XML automáticamente para métodos y propiedades, basándose en su tipo, parámetros, nombre y otra información contextual. Es un *Add-in* que además desinstala automáticamente versiones anteriores y las actualiza con las nuevas que estemos instalando.

Para buscar nuevas extensiones y actualizaciones en Visual Studio 2019 vamos al menú Extensiones -> Administrar Extensiones



Una vez aquí vamos a elegir En línea del menú de la izquierda y en el cuadro Buscar en Visual Studio Marketplace escribimos GhostDoc. Nos muestra el siguiente resultado.

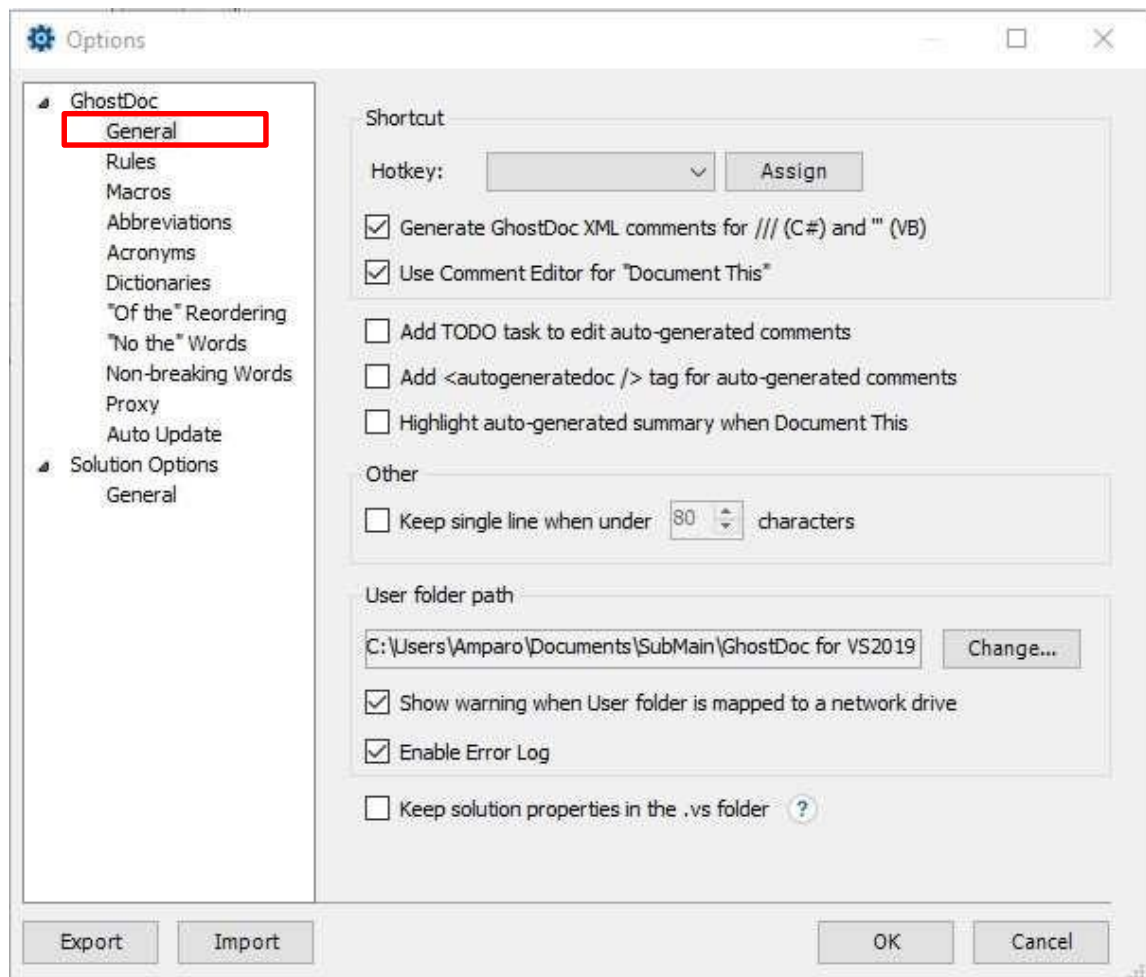
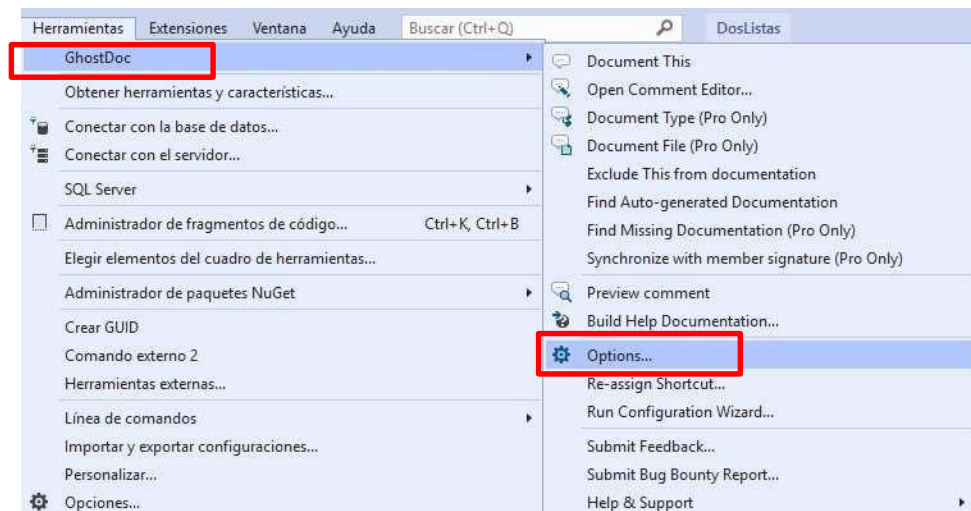


Pulsamos en Descargar y realizamos la instalación de GhostDoc.

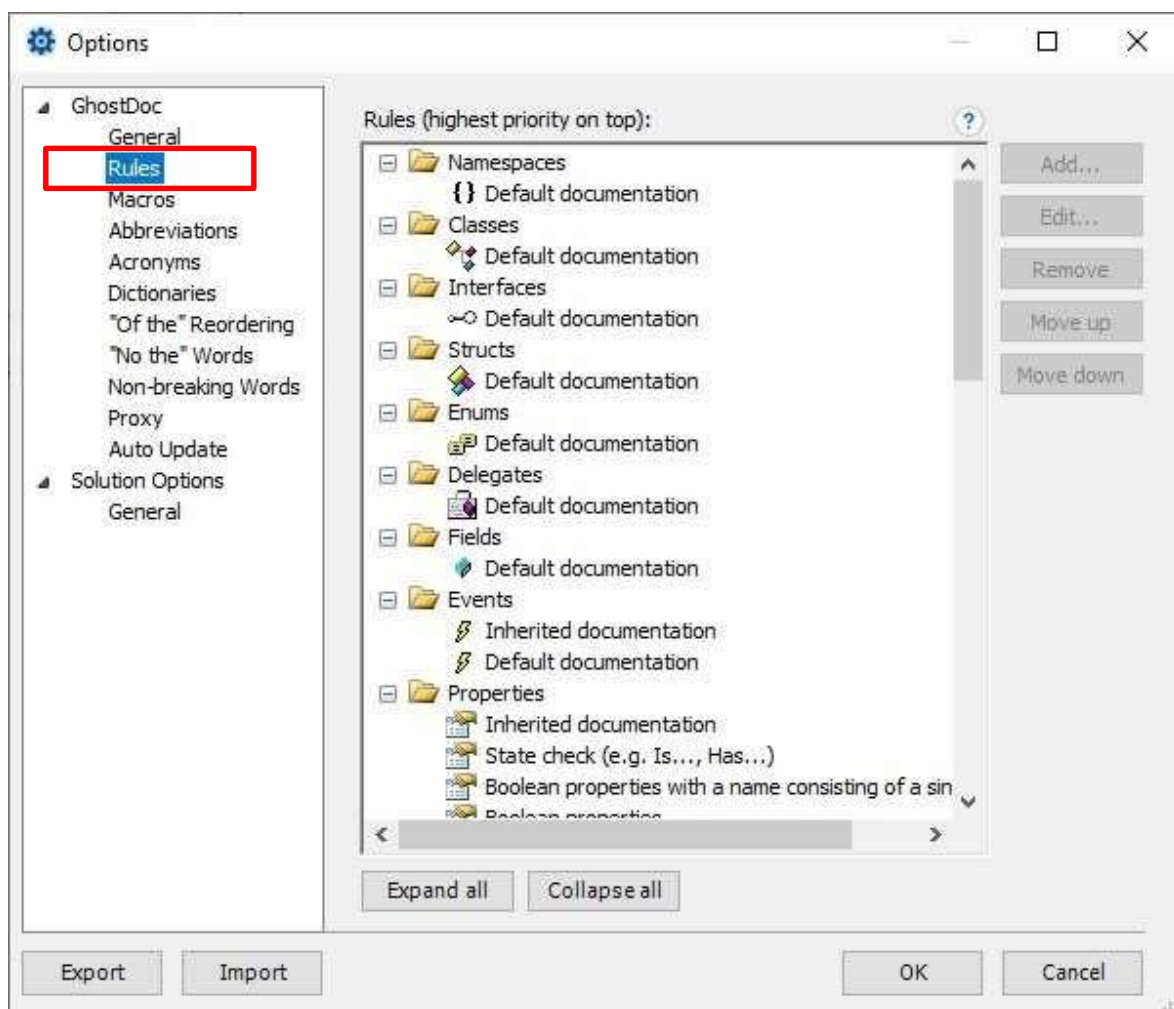
3.3.1. Configuración de GhostDoc.

En primer lugar, vamos a ver las opciones de configuración de GhostDoc.

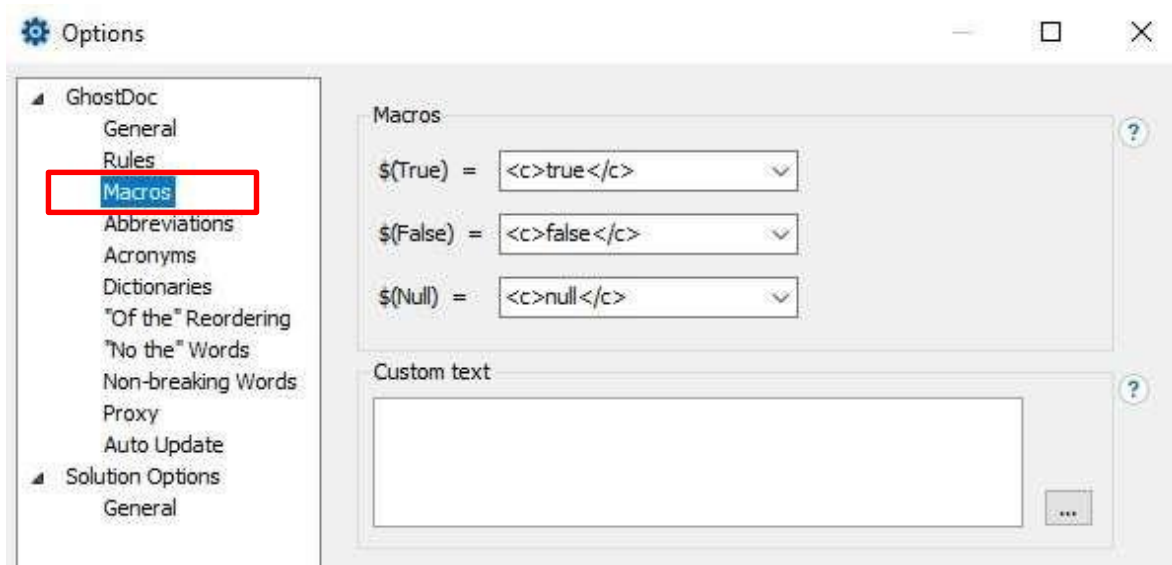
En las opciones generales podemos ver:

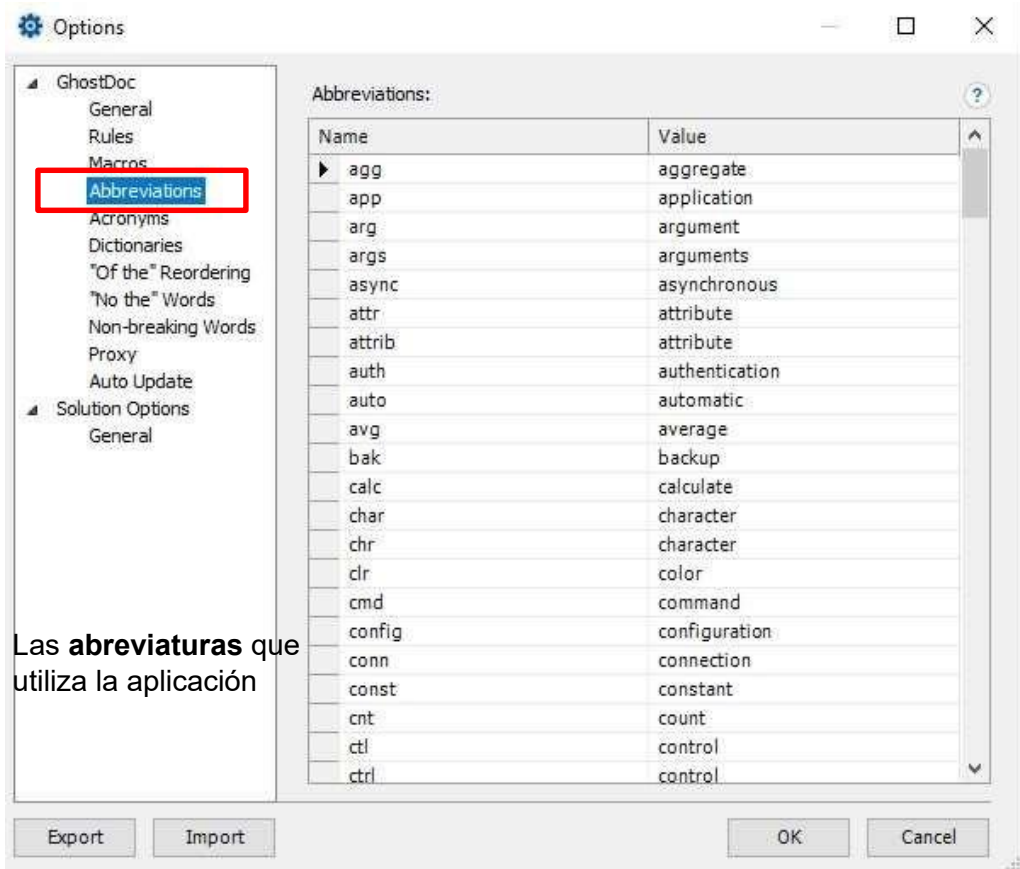


También tenemos una serie de reglas, que podemos editar. Además, se pueden añadir reglas nuevas.

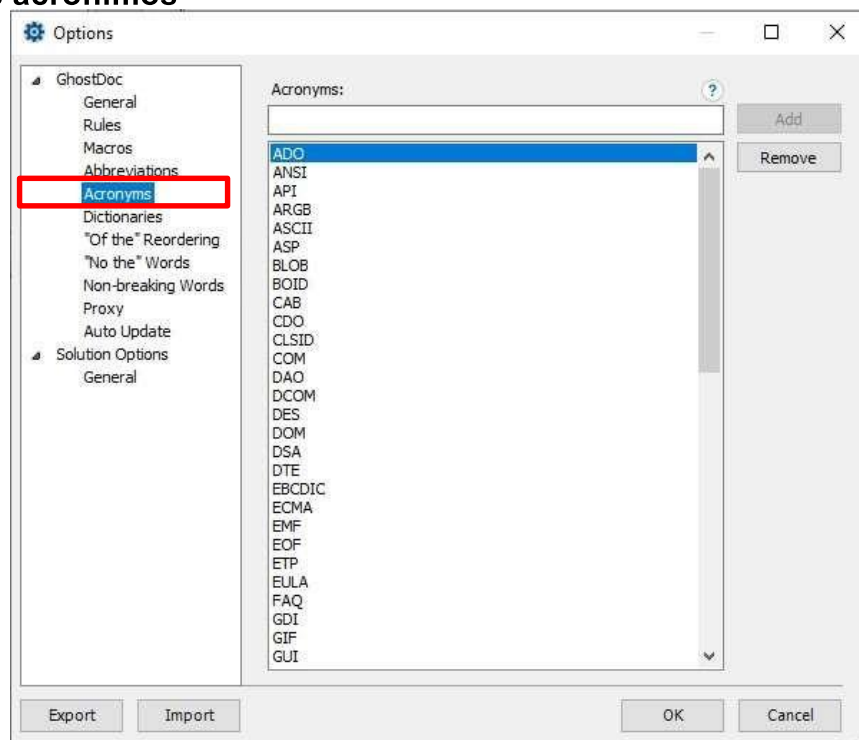


También tenemos Macros

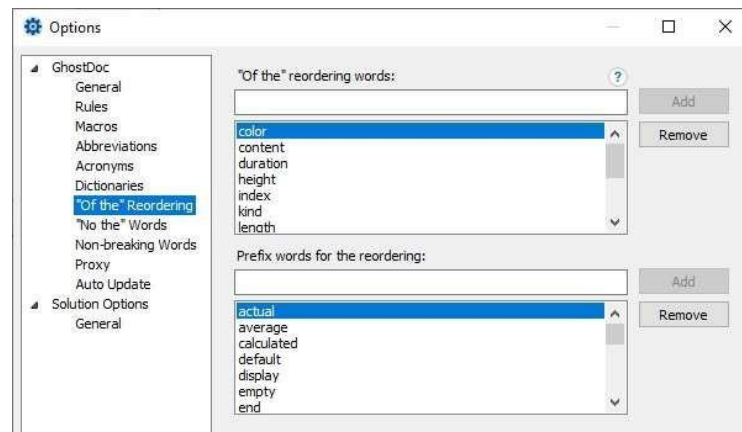




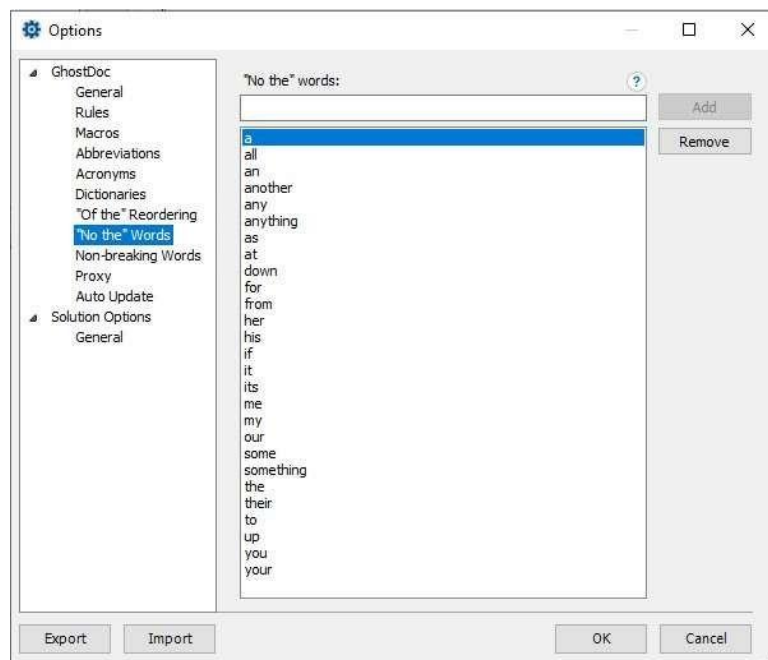
Los acrónimos



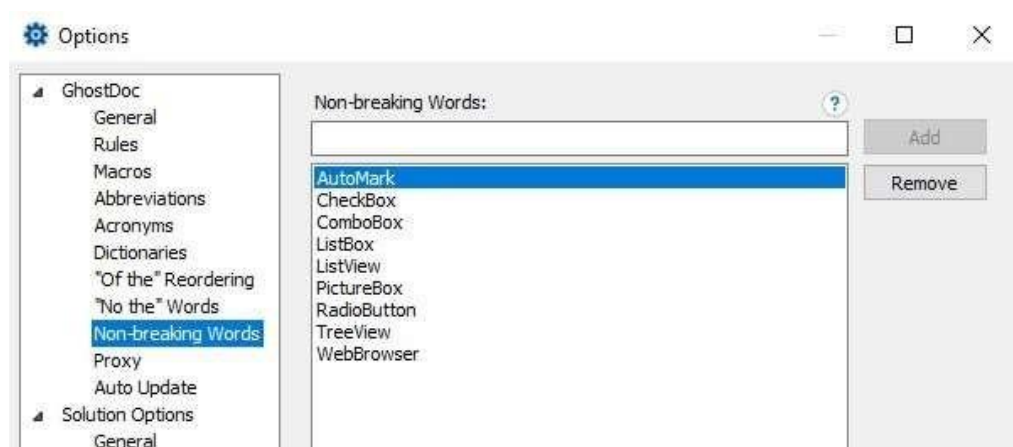
Las palabras de reordenación “Of the”:



Las palabras que no deben ir precedidas por “the”:

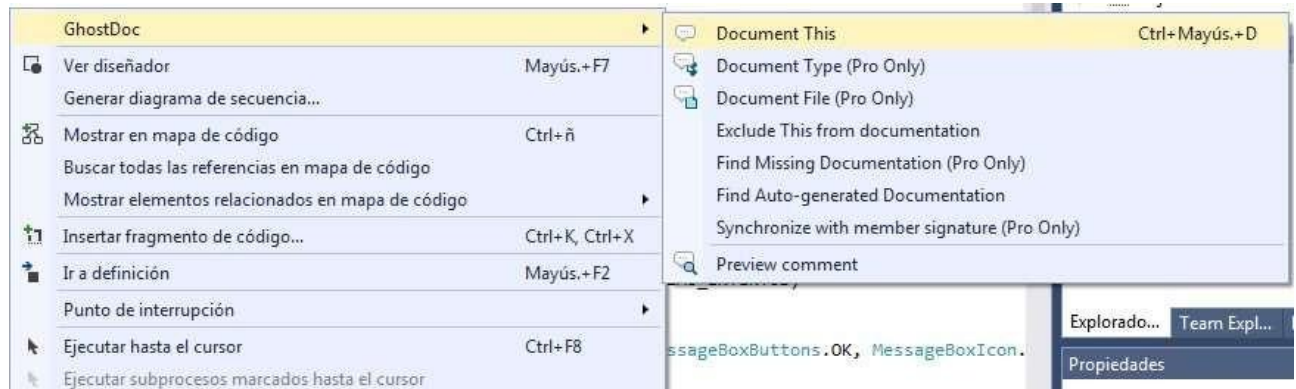


Las palabras que no se separaran al escribir los comentarios:



3.3.2. Generar documentación con GhostDoc.

Veamos cómo se documenta el código utilizando GhostDoc. Hacemos clic con el botón derecho sobre el código que queremos documentar y elegimos GhostDoc -> Document This



Y en este caso se genera lo siguiente:

```
''' <summary>
''' Handles the Click event of the btn_Conectar control.
''' </summary>
''' <param name="sender">The source of the event.</param>
''' <param name="e">The <see cref="System.EventArgs"/> instance containing the event data.</param>
'''
References
Private Sub btn_Conectar_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
```

En estos comentarios que se han añadido a nuestro código podemos introducir los comentarios que nosotros deseemos. Por ejemplo:

```
''' <summary>
''' Handles the Click event of the btn_Conectar control.
''' Esta es una muestra de utilización de GhostDoc.
''' </summary>
''' <param name="sender">The source of the event.</param>
''' <param name="e">The <see cref="System.EventArgs"/> instance containing the event data.</param>
```

3.4. Herramienta SandCastle.

SandCastle es una herramienta creada por Microsoft y publicada Open Source en Codeplex para crear documentación estilo MSDN basándose en los assemblies .NET y sus comentarios XML asociados.

Es una herramienta basada en línea de comandos y no tiene una GUI predefinida. Afortunadamente la aplicación SandCastle Help Builder ha sido construida sobre SandCastle para dotar de un fácil manejo de todas las opciones que tiene SandCastle.

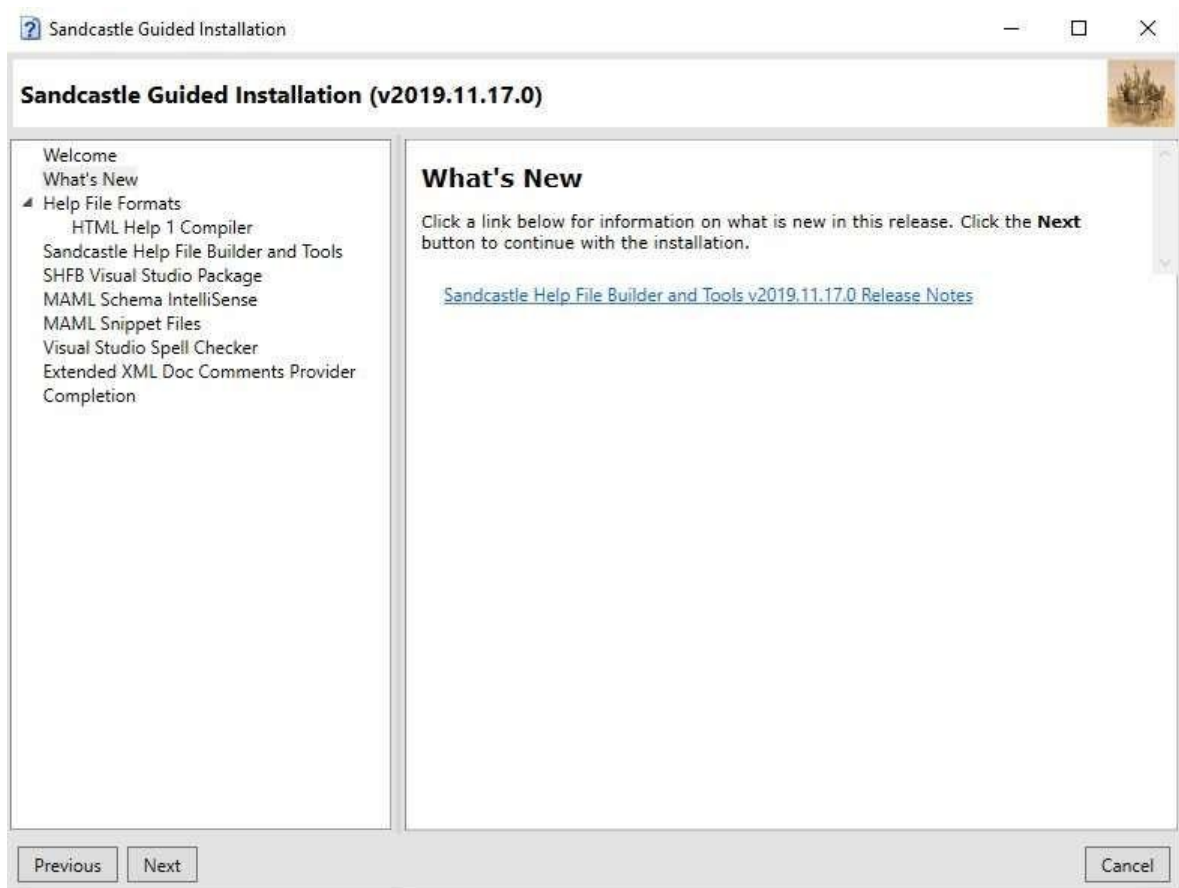
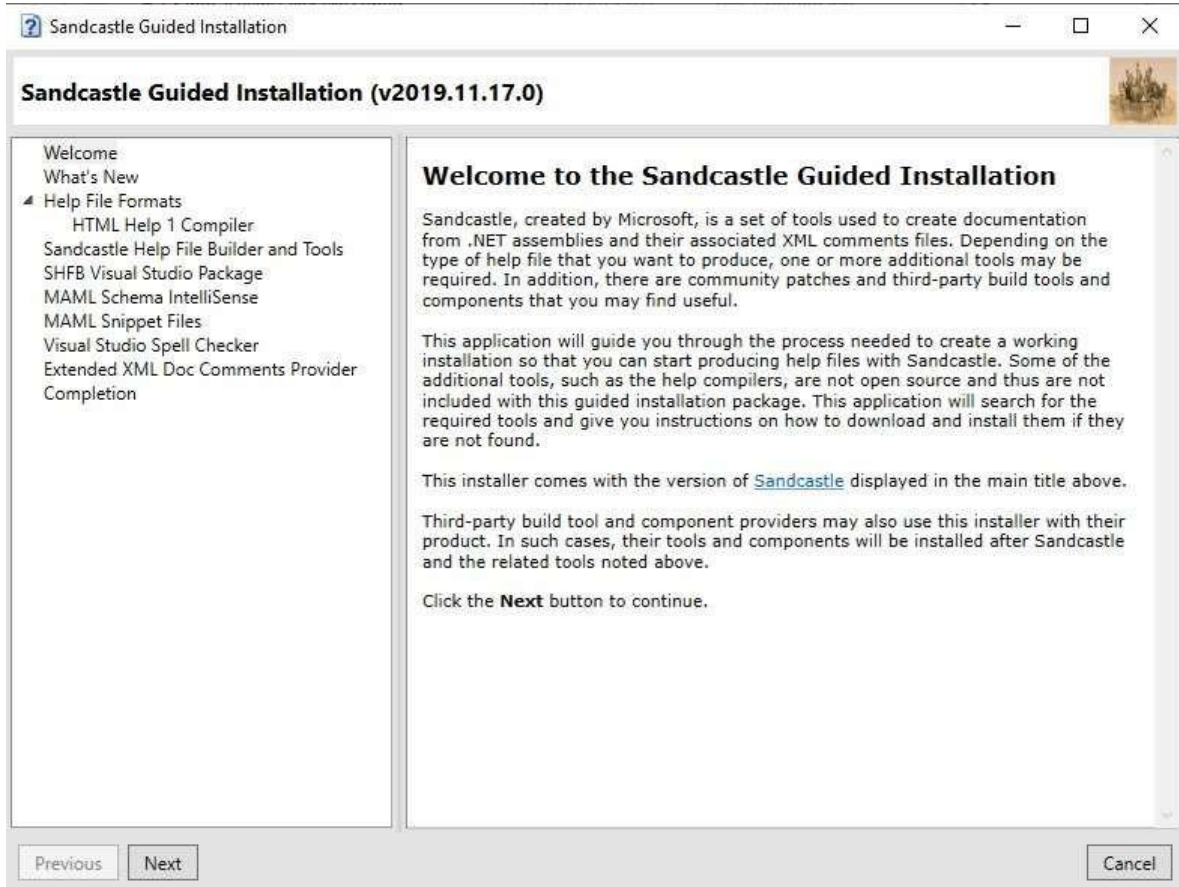
3.4.1. Instalación de SandCastle Help File Builder.

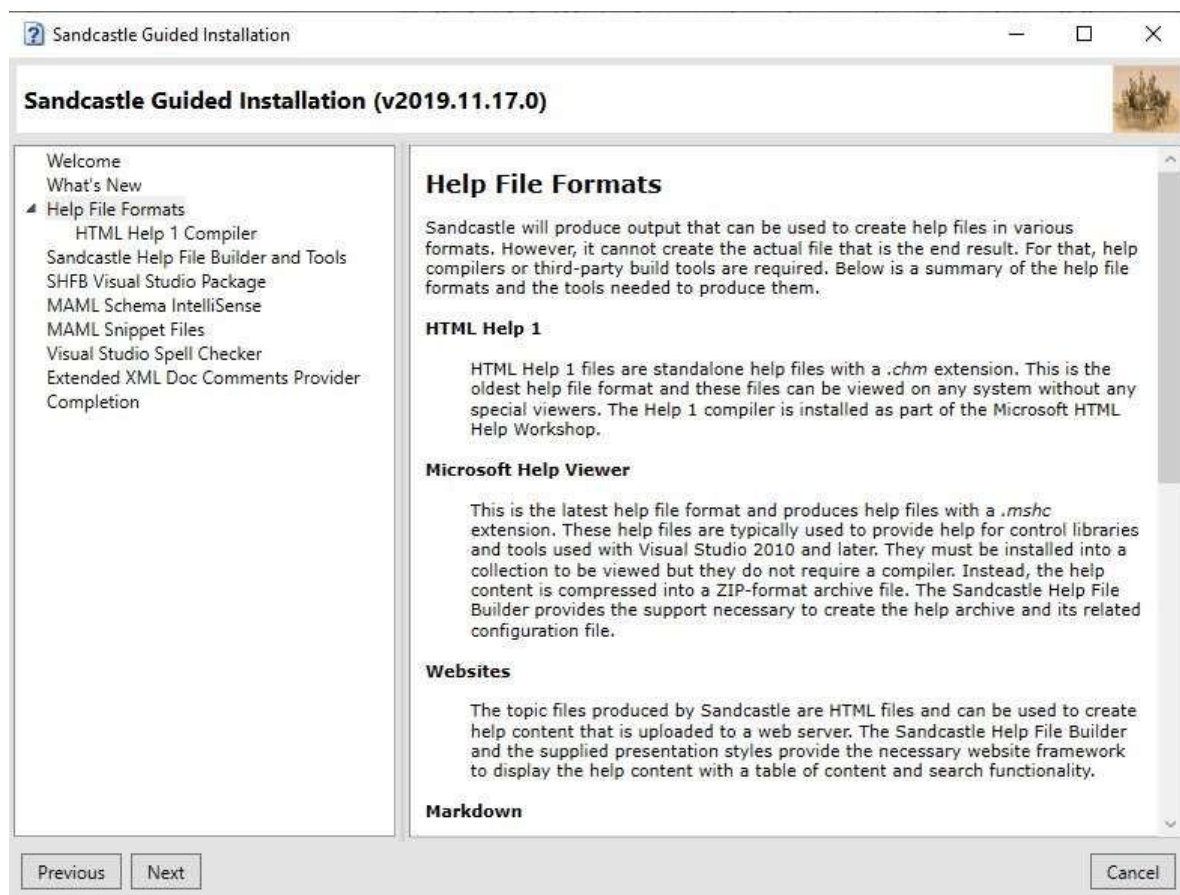
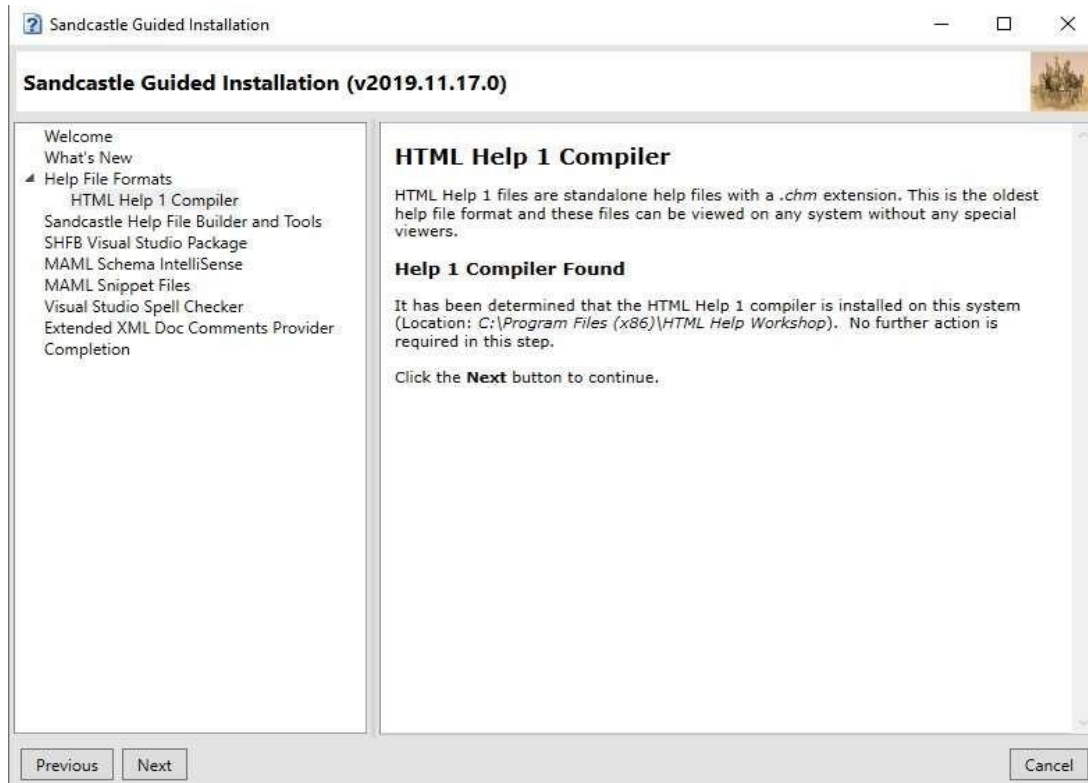
La instalación de esta herramienta permite disfrutar de un entorno más amigable e intuitivo para la creación del archivo de ayuda.

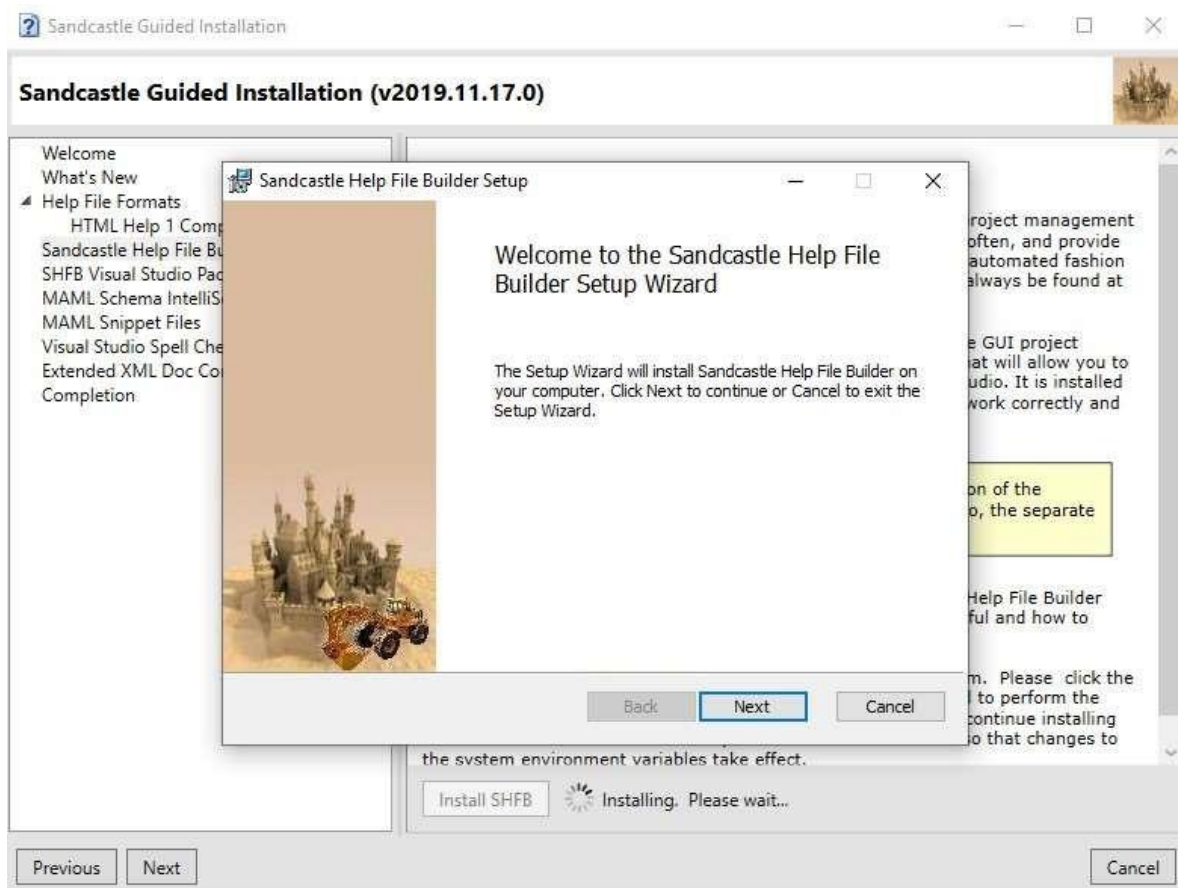
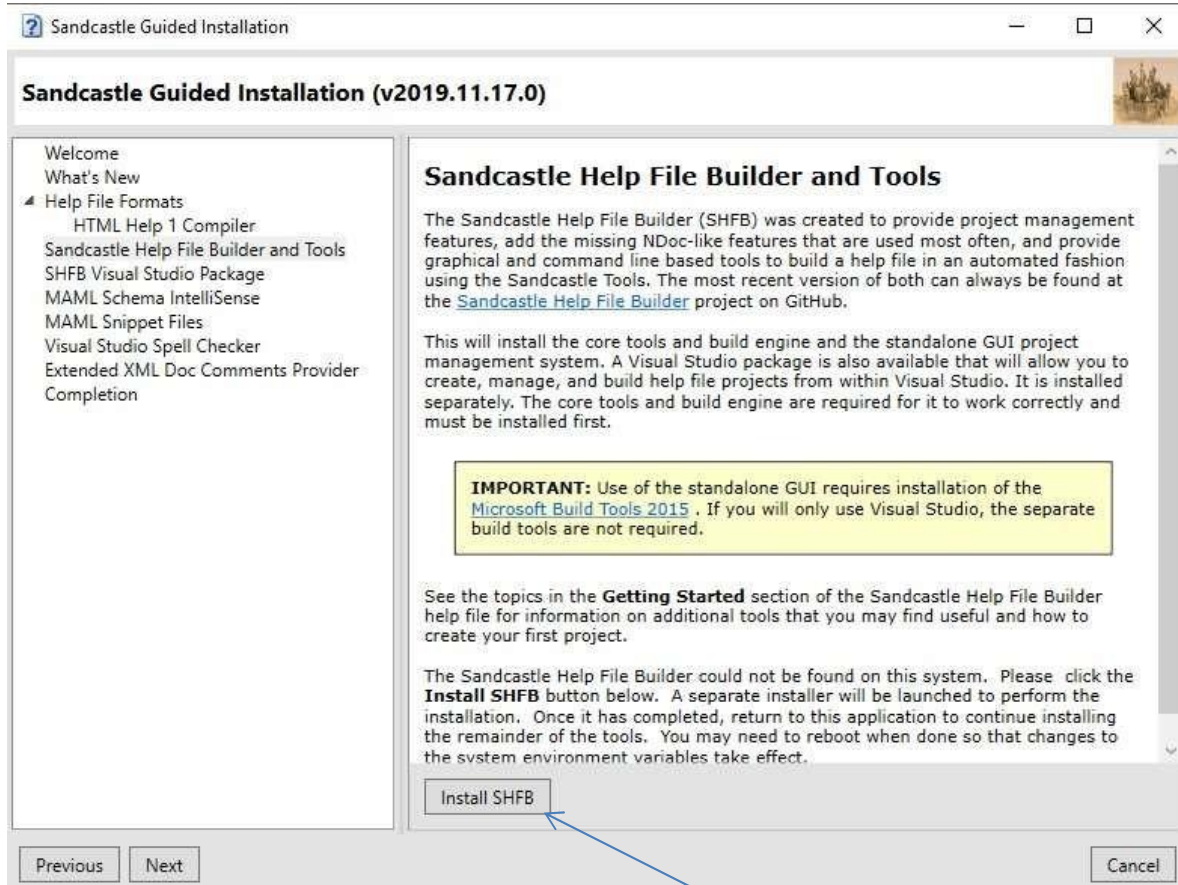
Descargamos SandCastle Help File Builder de la siguiente dirección (obtendremos un zip con la versión actualizada):

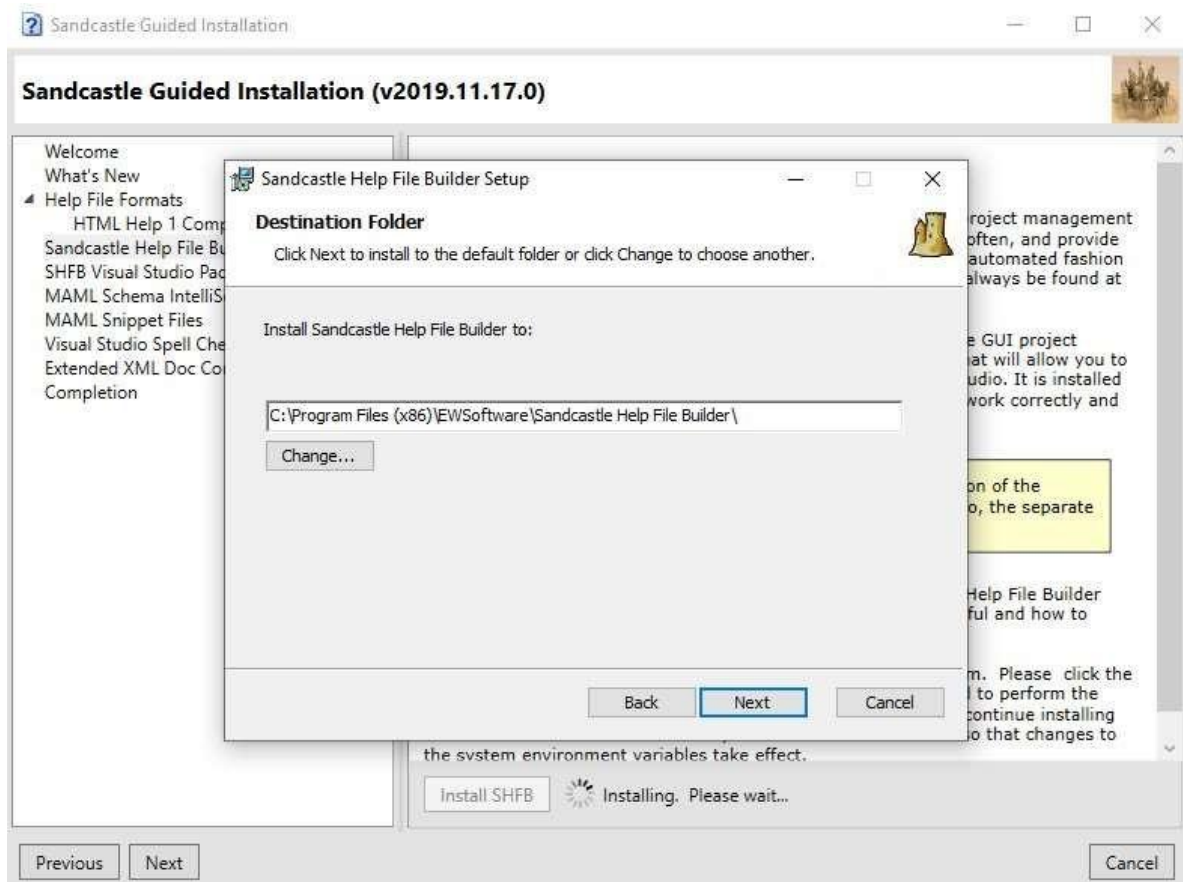
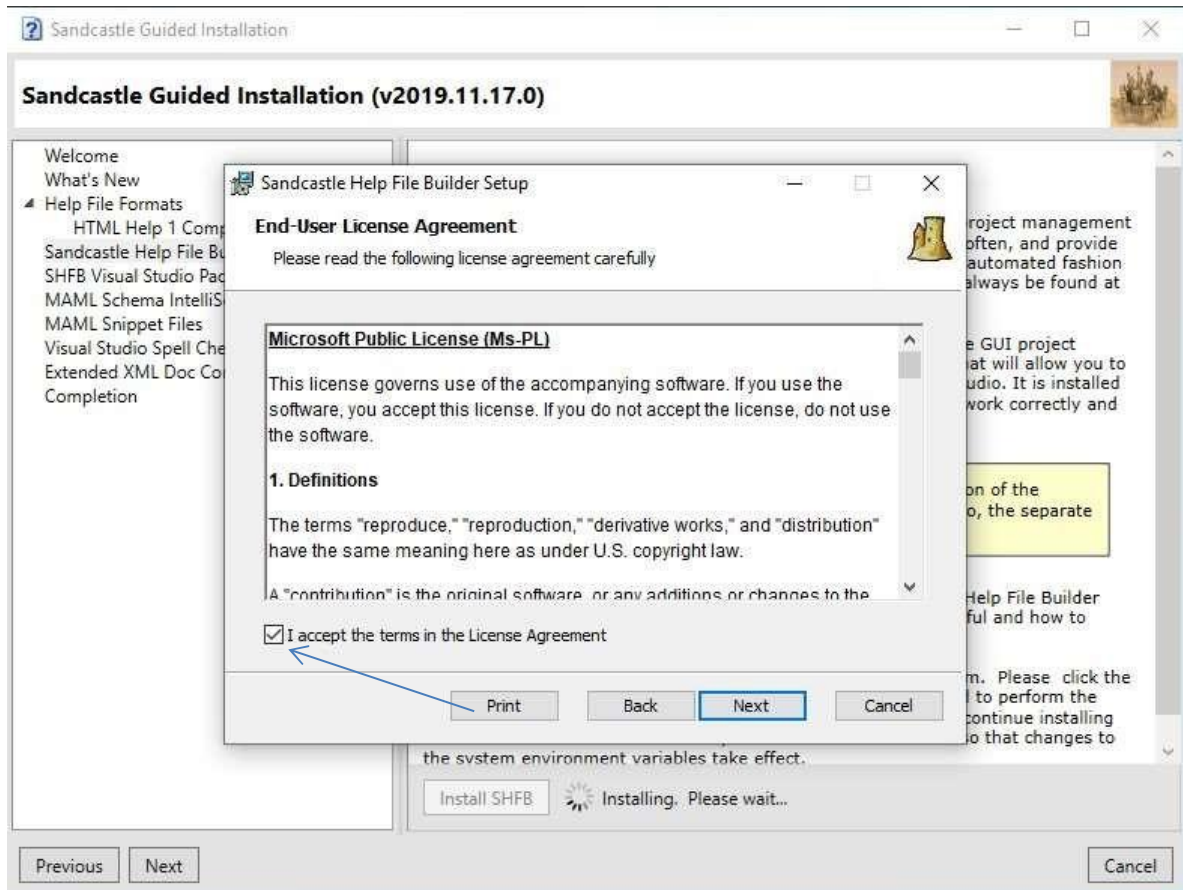
- <https://github.com/EWSoftware/SHFB/releases>

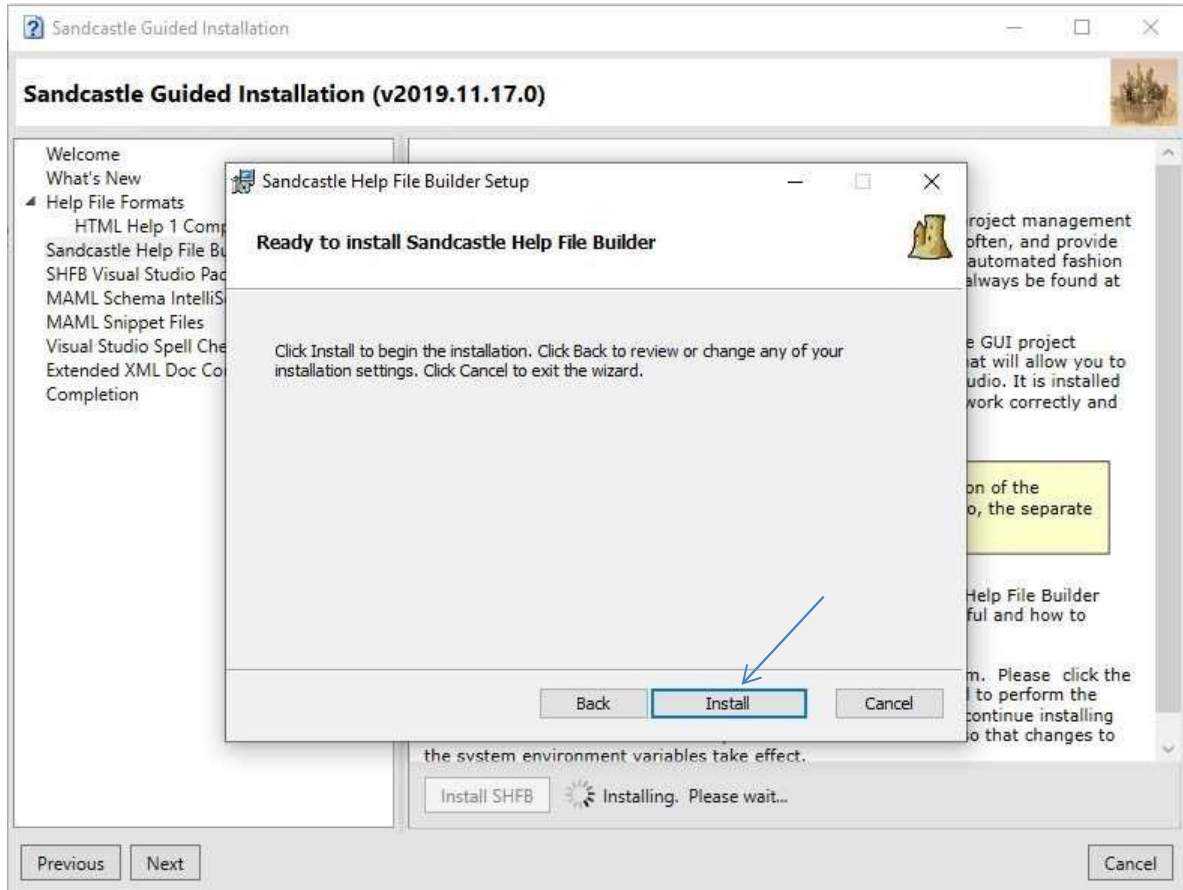
Comenzamos la instalación:

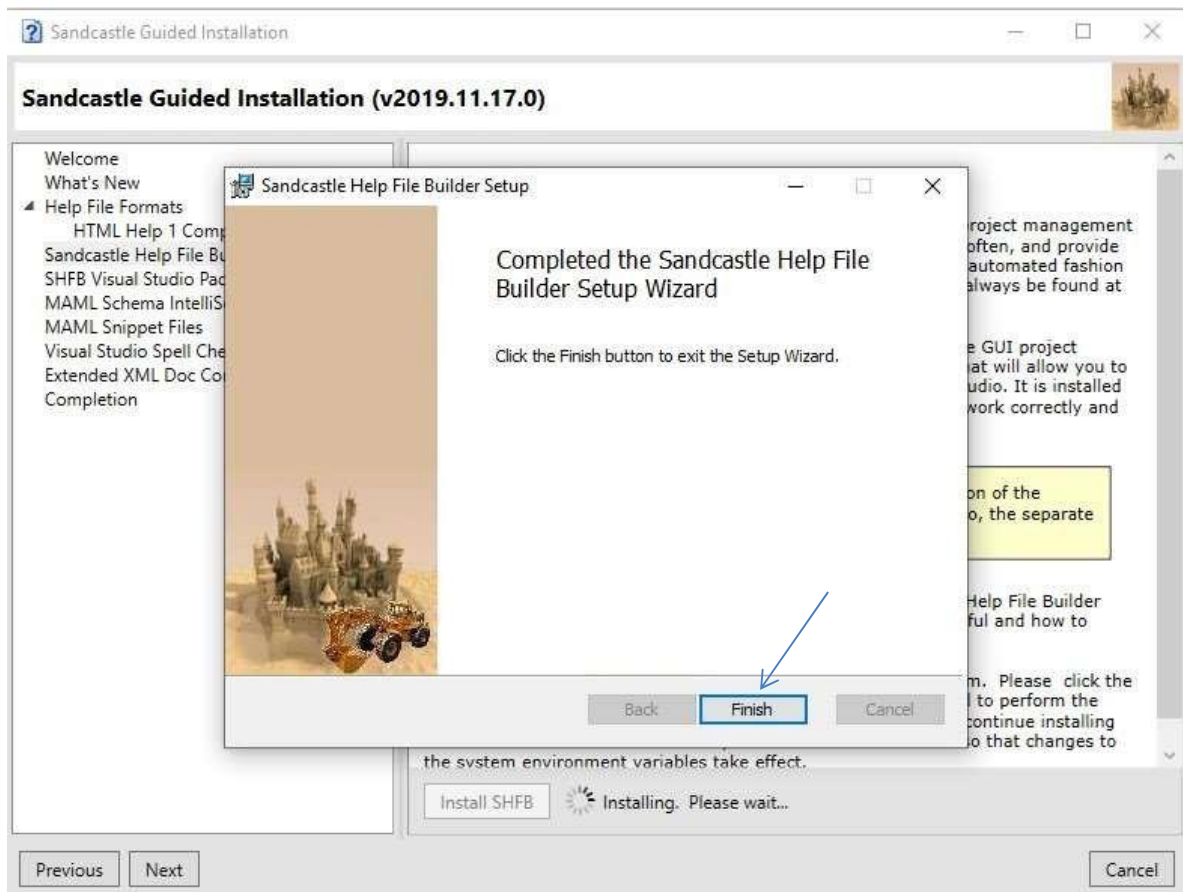
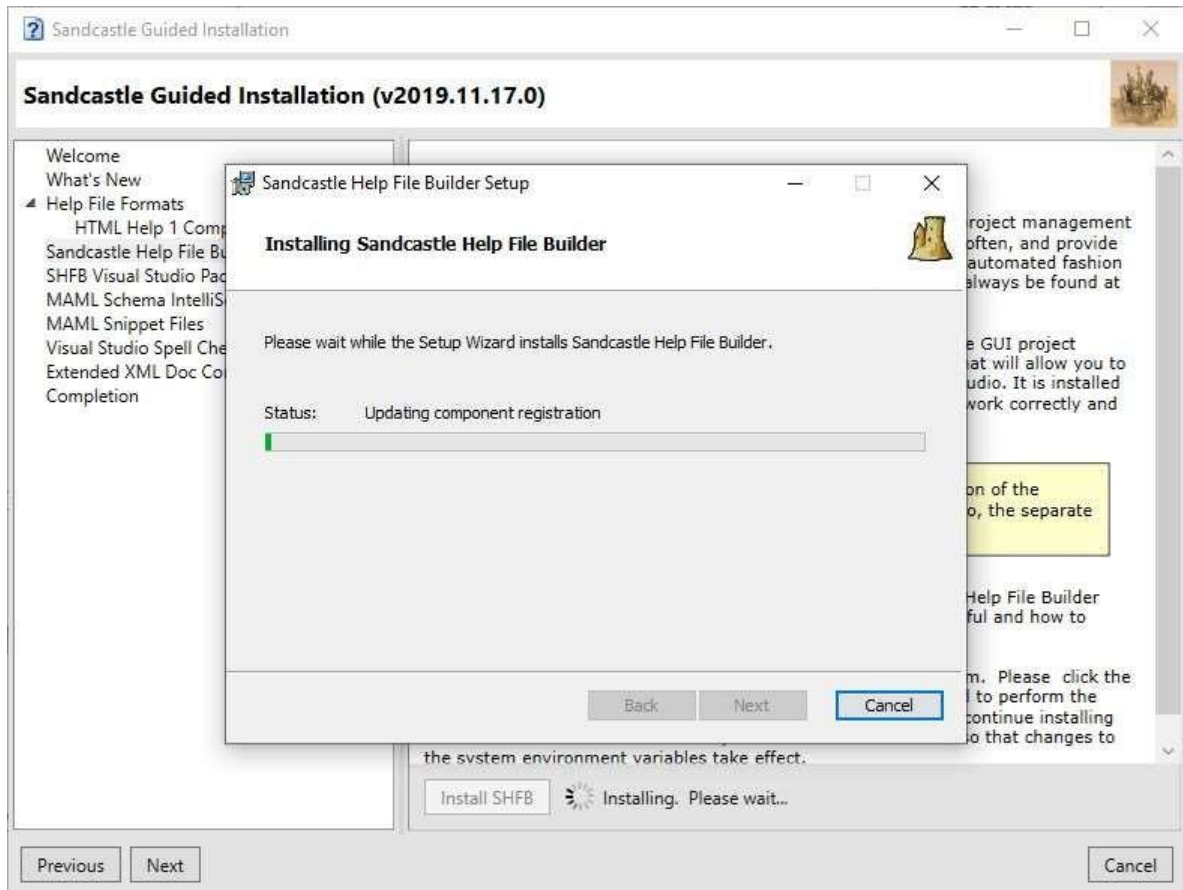


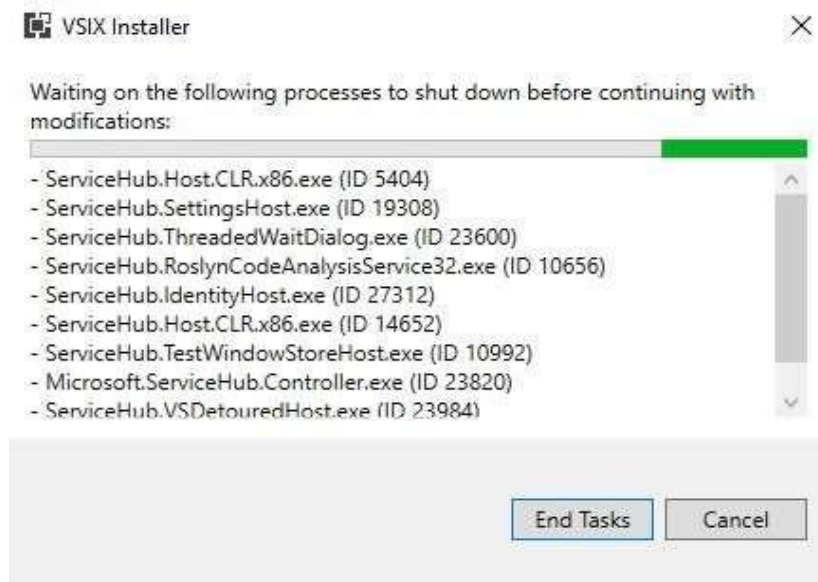
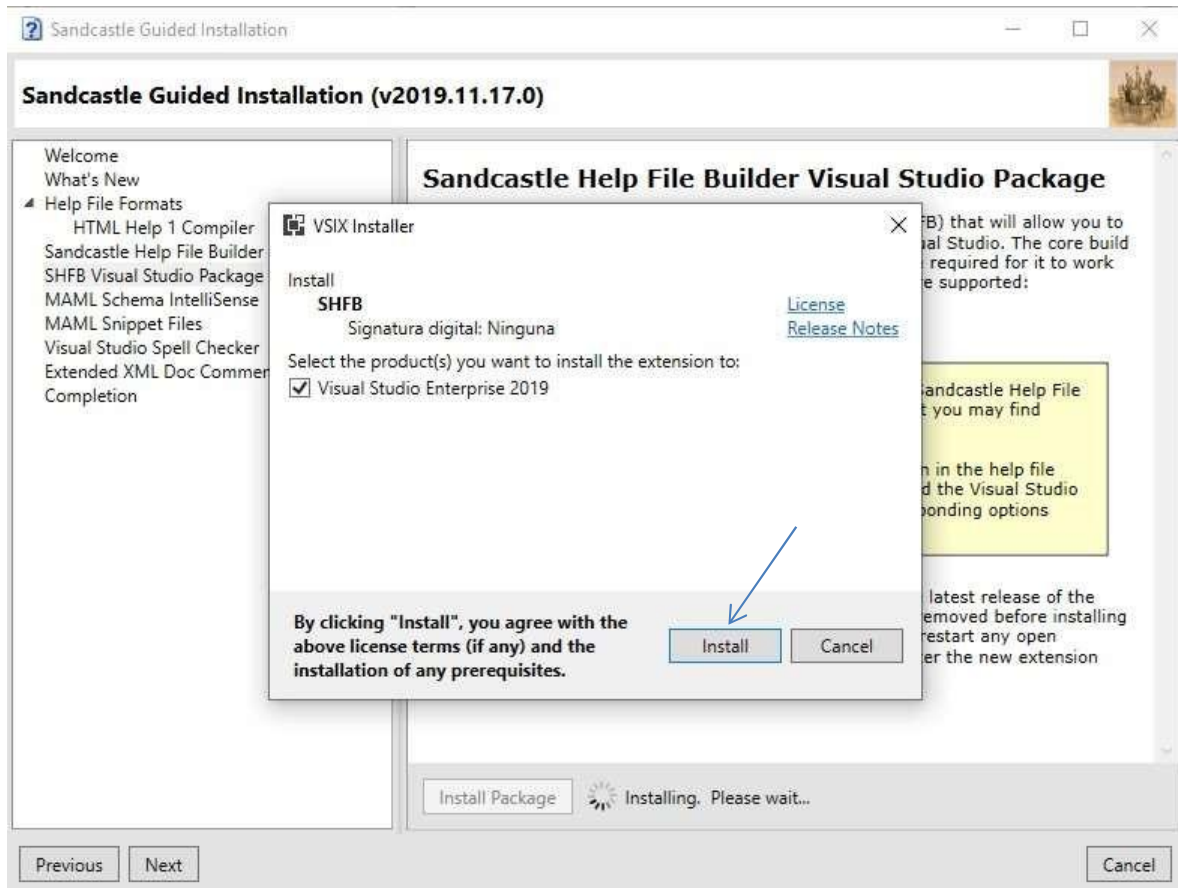




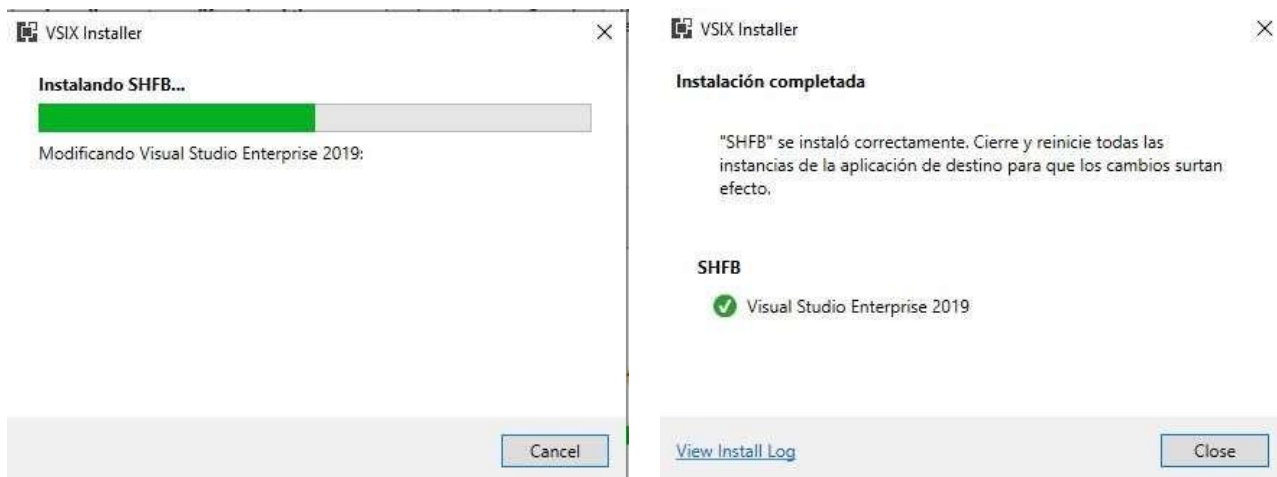




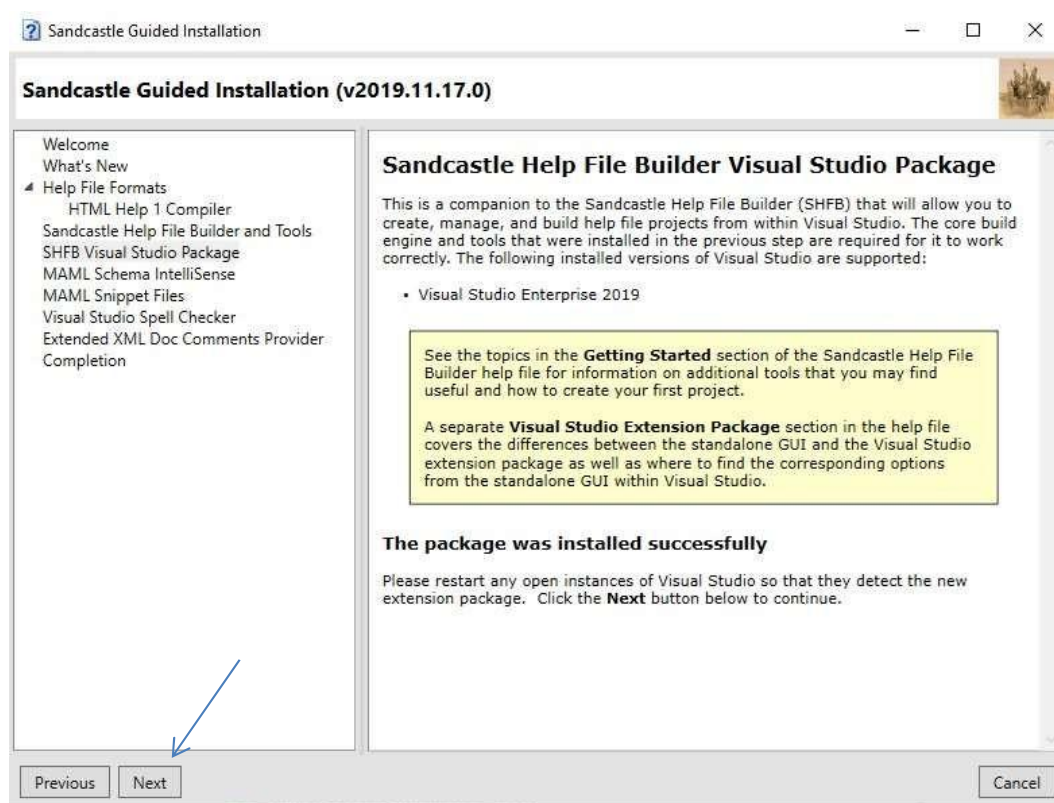


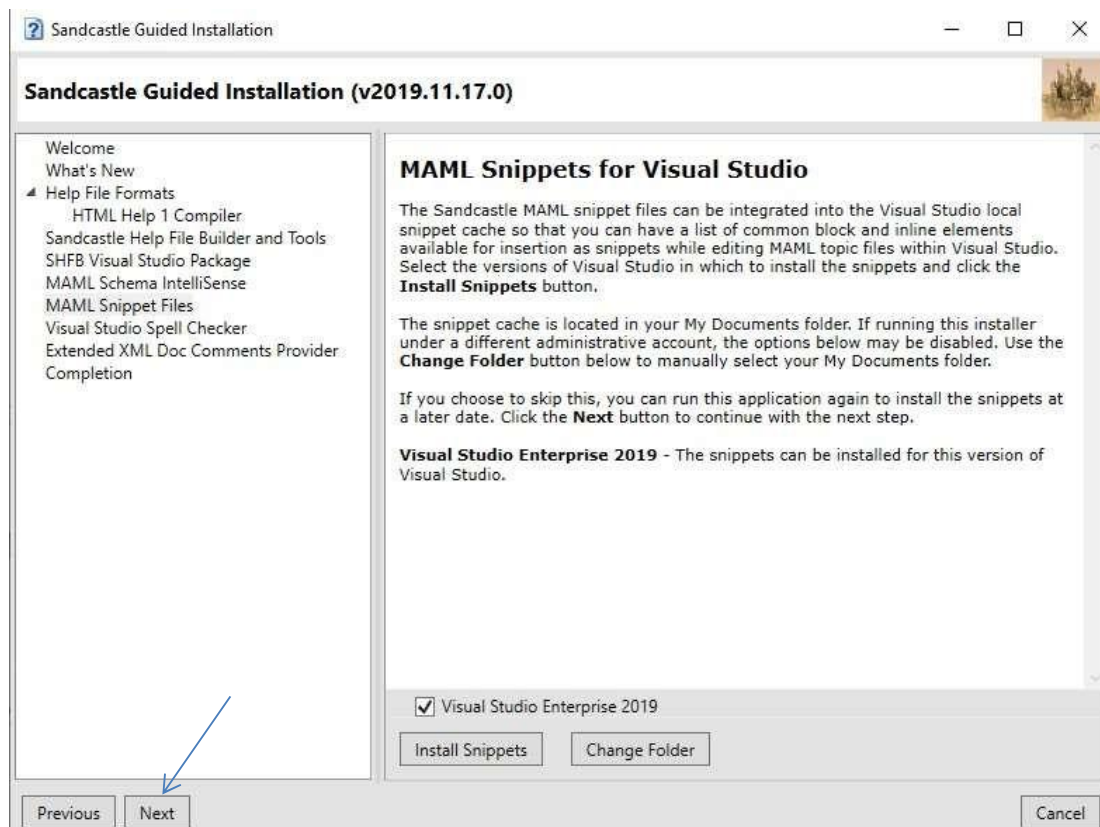
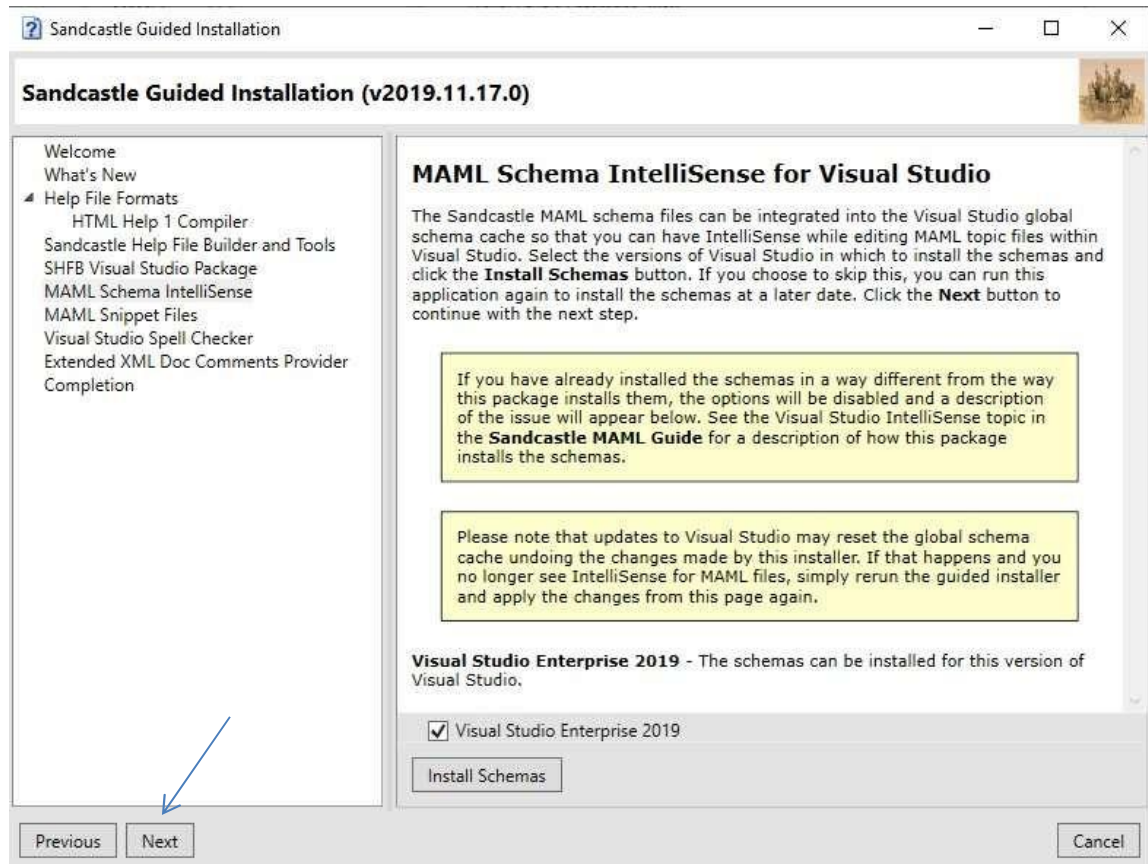


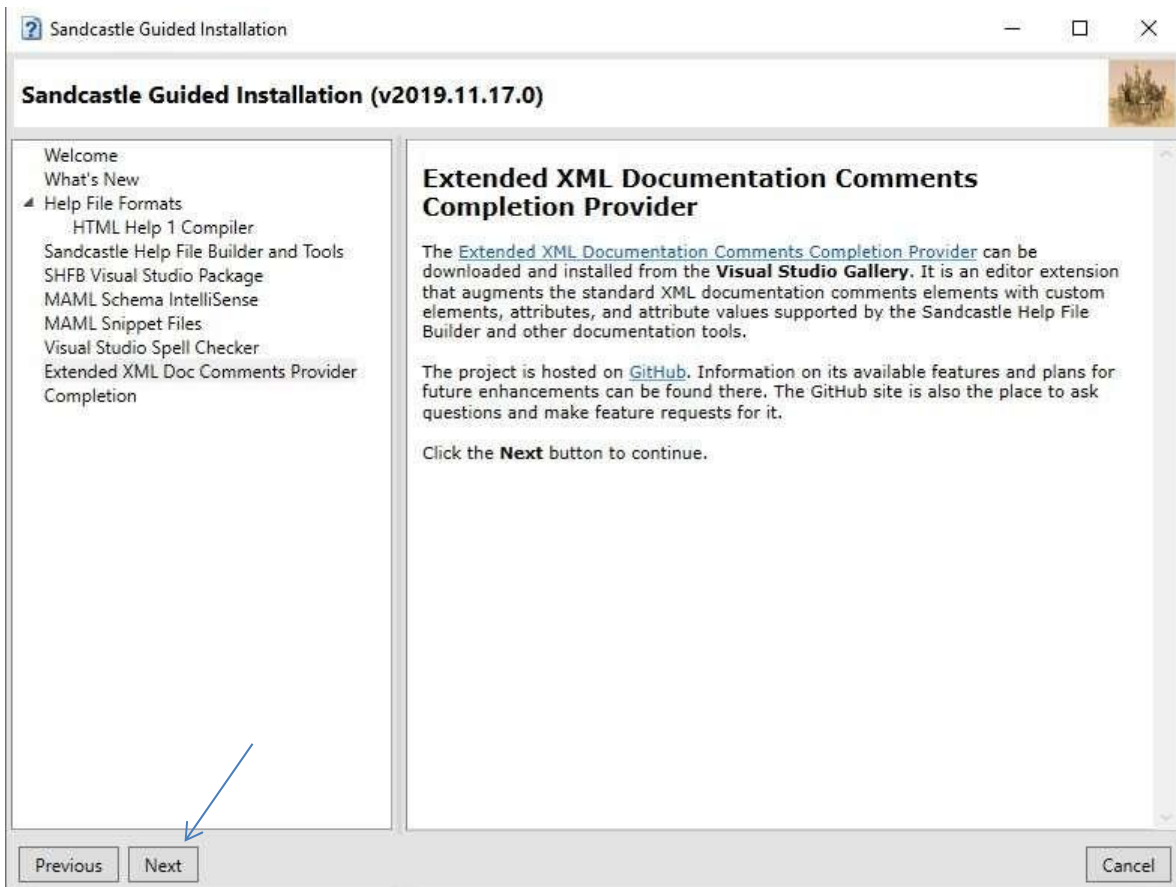
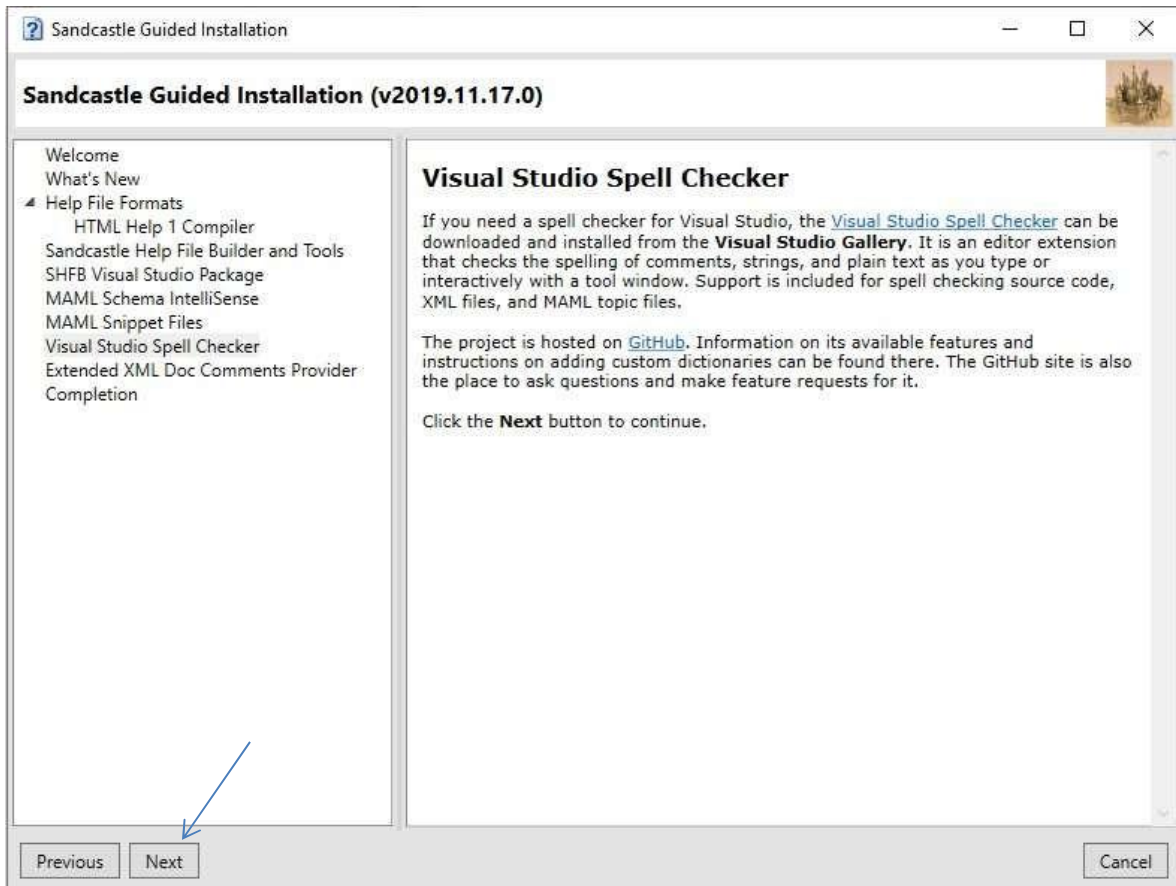
Debemos cerrar Visual Studio, si lo tenemos abierto, para que continúe el proceso de instalación.

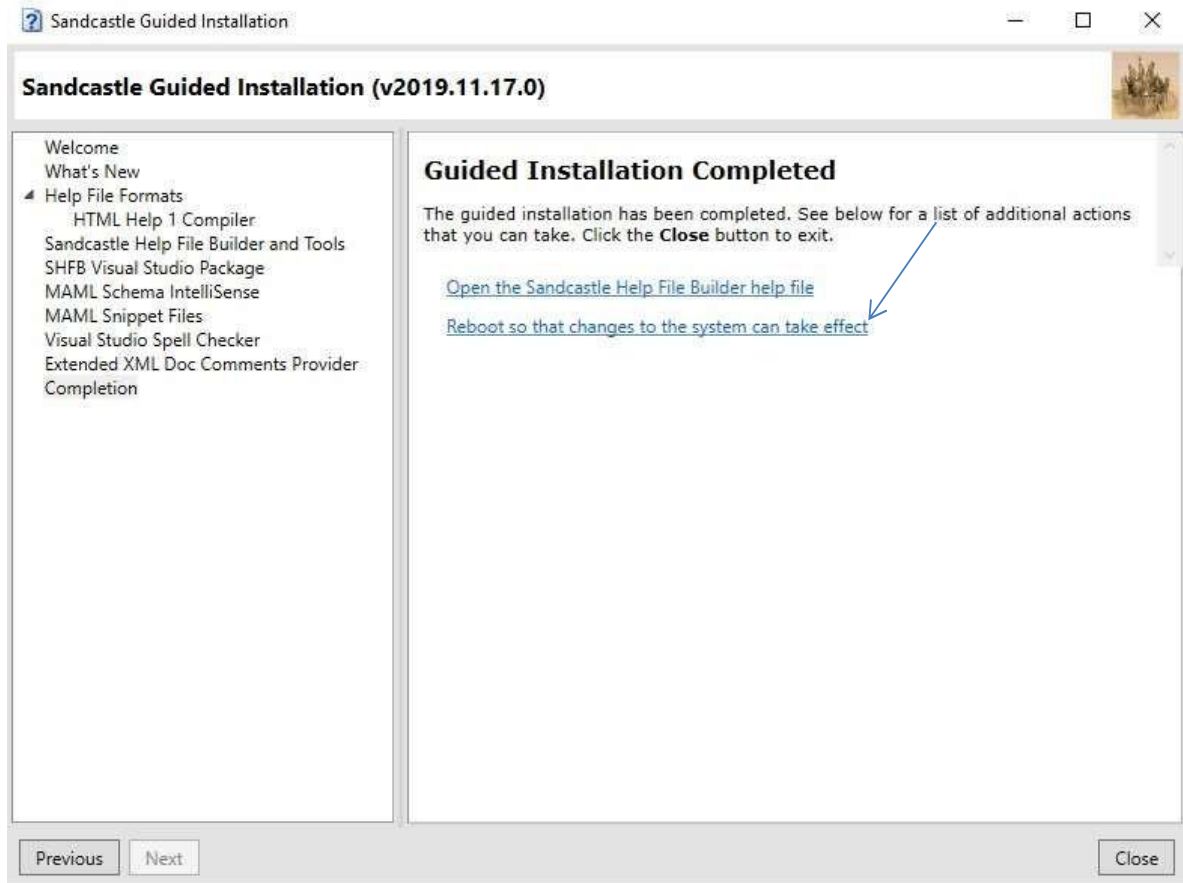


Después de pulsar en Close, llegamos a la siguiente ventana:



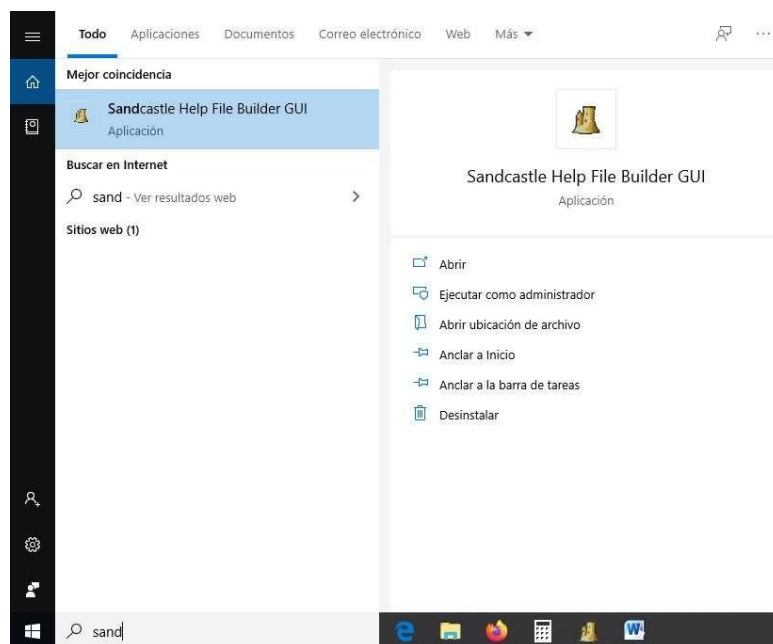




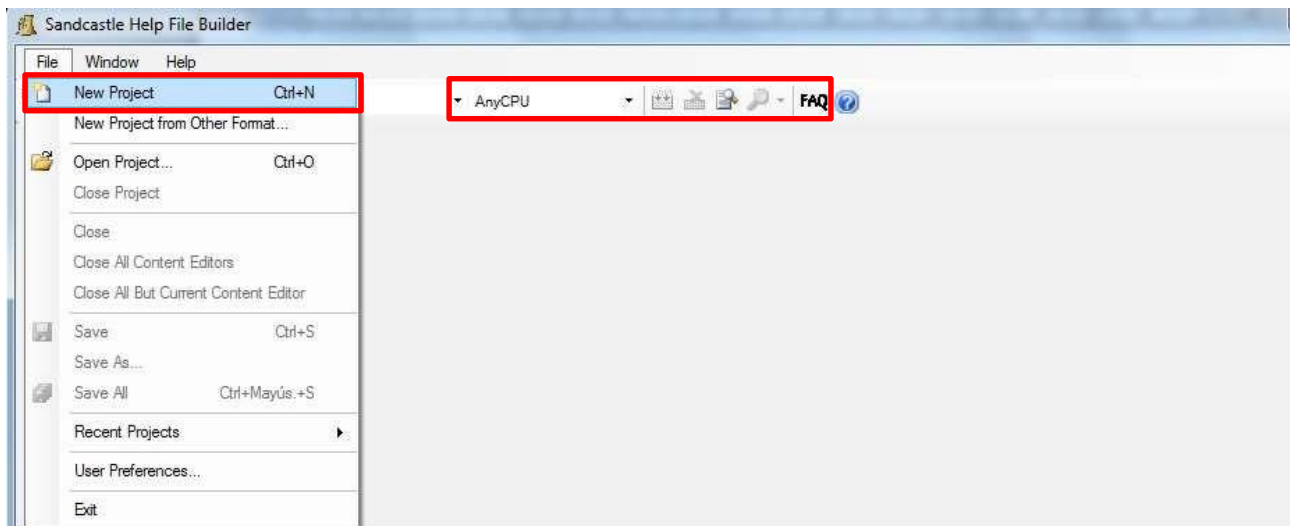


3.4.2. Generar documentación con SandCastle.

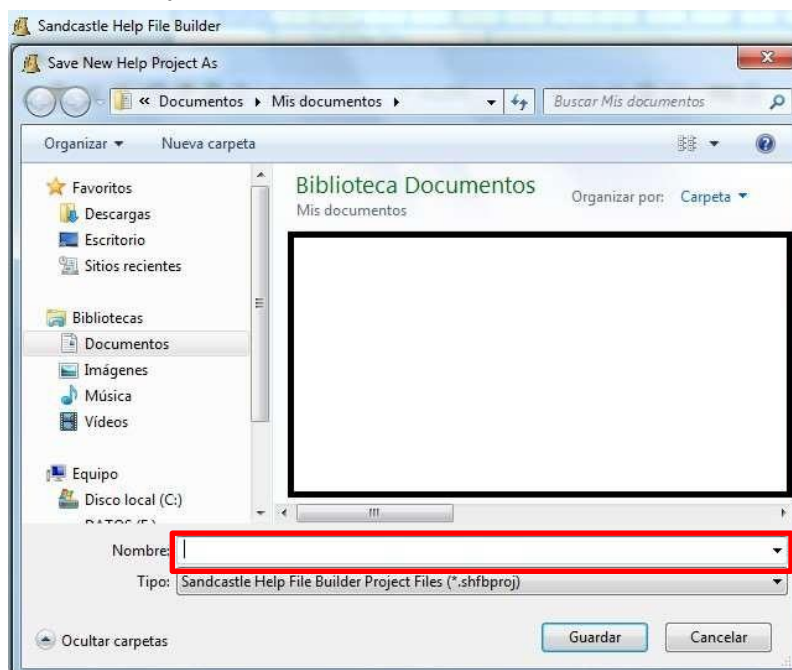
Una vez instalado SandCastle Help File Builder GUI podemos lanzar la aplicación.



Para generar un archivo de ayuda crearemos un nuevo proyecto en SandCastle Help File Builder.
File -> New Project

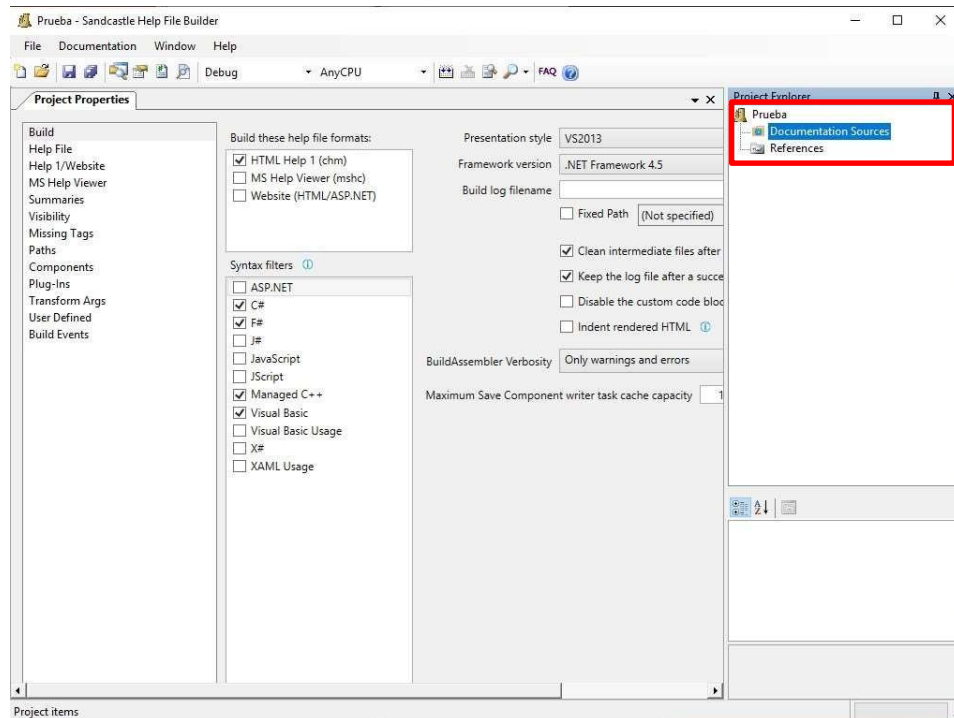


Cuando creamos un nuevo proyecto tenemos que darle nombre:



En el apartado **Project Explorer**:

1. Botón derecho sobre **Documentation Sources**.
2. Selecciona: **Add -> Documentation Sources...**



El proyecto solicitará al usuario la inclusión del origen de dónde crear la documentación, ya sean ensamblados, librerías, archivos de proyecto o solución. La herramienta hará uso de toda la información que pueda extraer de éstos, como los comentarios de código, para generar el documento de ayuda buscado.

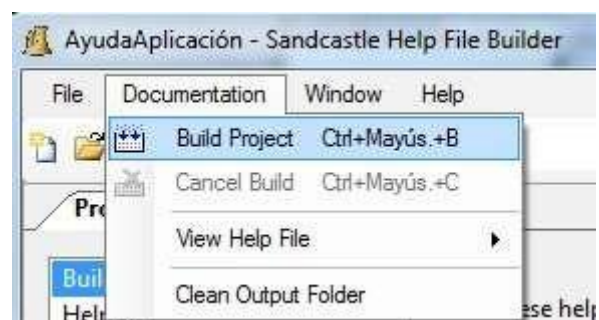
Podemos seleccionar la solución del proyecto que vamos a documentar, Projectname.sln. o acceder a la carpeta bin\Release del proyecto y seleccionar el archivo XML.

Optaremos por la segunda opción.

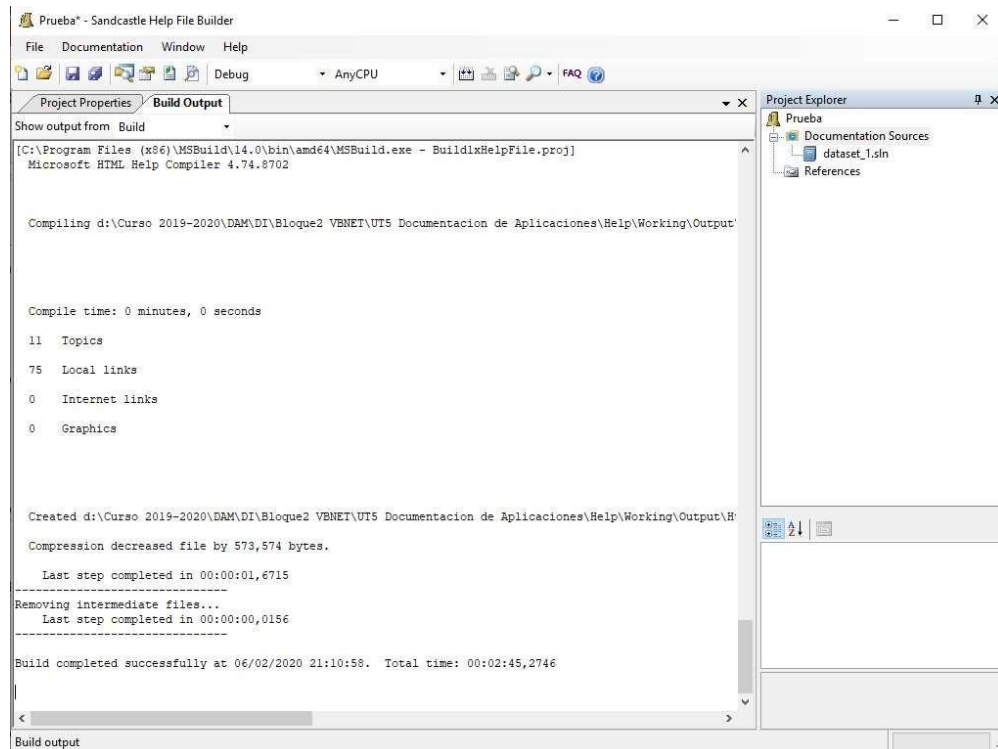
Deberemos repetir estos pasos por cada proyecto que deseemos incluir en la documentación.

A continuación, para generar el proyecto de documentación deberemos seguir los siguientes pasos:

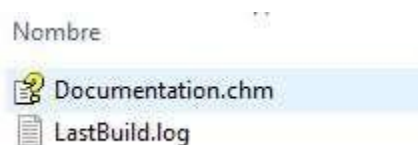
- Antes, en el apartado Project Properties, configurar las opciones de personalización para la generación de la documentación.
- Compilar el proyecto (Documentation -> Build Project).



Tras terminar la compilación, se muestra una pantalla como esta:

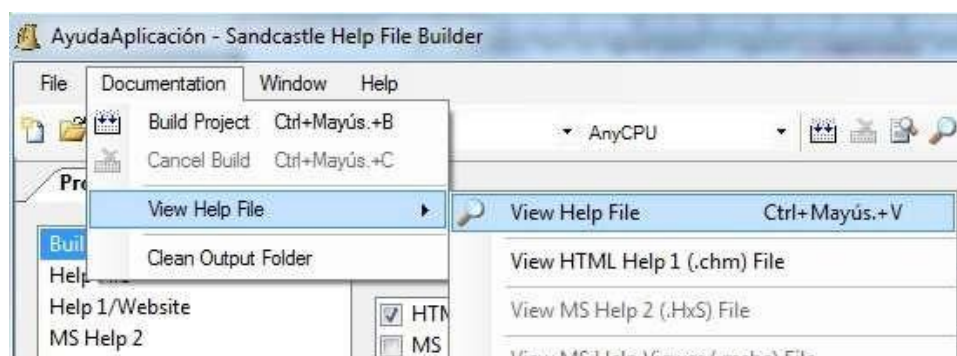


Recuerda que el resultado de la compilación se guardará en la carpeta Help en la ruta donde hayas guardado el archivo de proyecto de SandCastle.

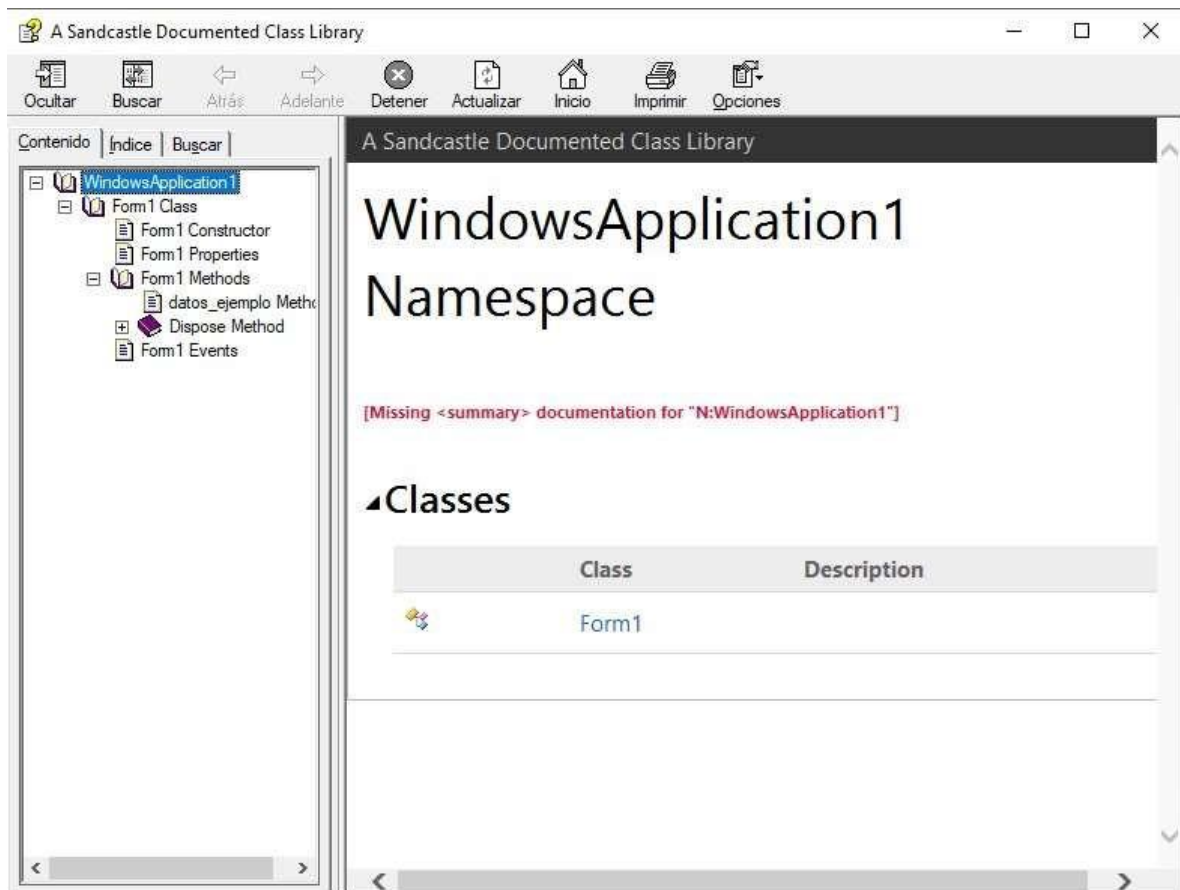


Debemos tener en cuenta que, para disponer de una documentación completa, los ensamblados deben estar debidamente comentados.

Finalmente, podemos ver la documentación generada por SandCastle desde Documentation -> View Help File -> View Help File



El resultado es el siguiente:



CONCLUSIÓN: Para documentar un proyecto de Visual Studio deberemos seguir los siguientes pasos:

1. Documentar el código con comentarios en XML. Esto genera el archivo XML del proyecto.
Podemos hacerlo manualmente o utilizando una herramienta de ayuda como GhostDoc.
2. Generar el fichero de ayuda del proyecto, que podremos incluir en el mismo. Esto lo haremos con una herramienta como SandCastle.

3.5 DocFX

DocFX es actualmente la herramienta que se está implantando por eficiente (desde línea de comandos) y por su compatibilidad con las nuevas tecnologías HTML. Sin embargo, Sandcastle sigue siendo el casi indiscutible en un nicho específico: Aplicaciones de Escritorio Offline (Windows Forms/WPF). Donde falla DocFX, por el momento, es en la generación de archivos CHM.

Enlaces oficiales para documentarse:

- <https://learn.microsoft.com/es-es/shows/on-dotnet/intro-to-docfx#time=12m42s>
- https://dotnet.github.io/docfx/?WT.mc_id=ondotnet-c9-cephilli

4. Herramienta HelpNDoc.

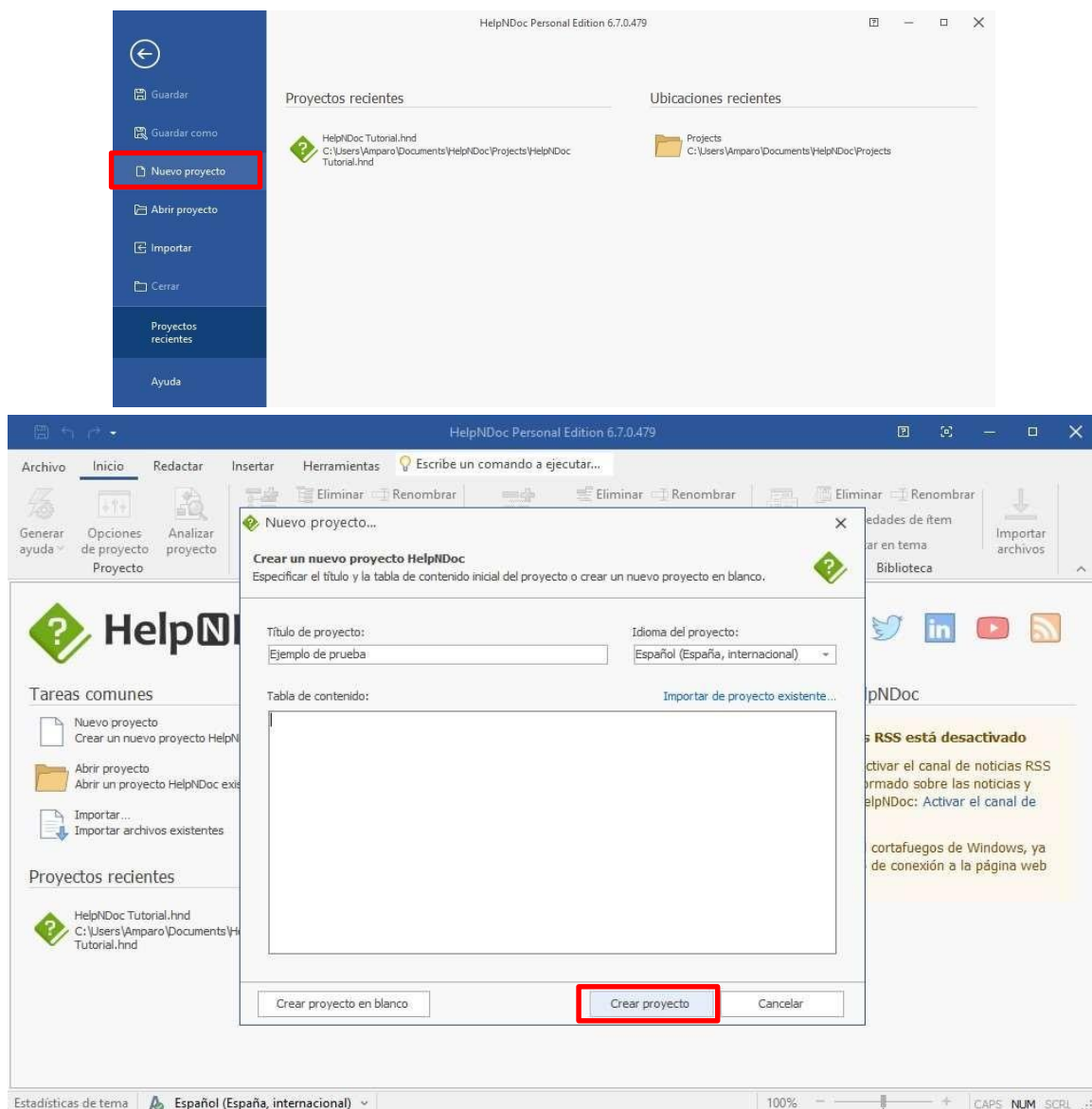
La herramienta HelpNDoc tiene una versión gratuita (que se puede descargar desde la dirección <http://www.helpndoc.com/download>) y permite crear archivos de ayuda en diferentes formatos (chm, PDF, HTML, etc.).

🔊 NO lee tu código fuente C#.

No es capaz de “escanear” las clases C#, extraer los comentarios `///` y crear una tabla automática con los métodos y parámetros. Por tanto, habrá que usar Sandcastle o DocFX para: Generar la Referencia de la API (documentación técnica interna para ti o futuros programadores). Usaremos HelpNDoc para: Generar la Guía del Usuario (para el cliente final). "Cómo interpretar el informe de Inventario Bajo".

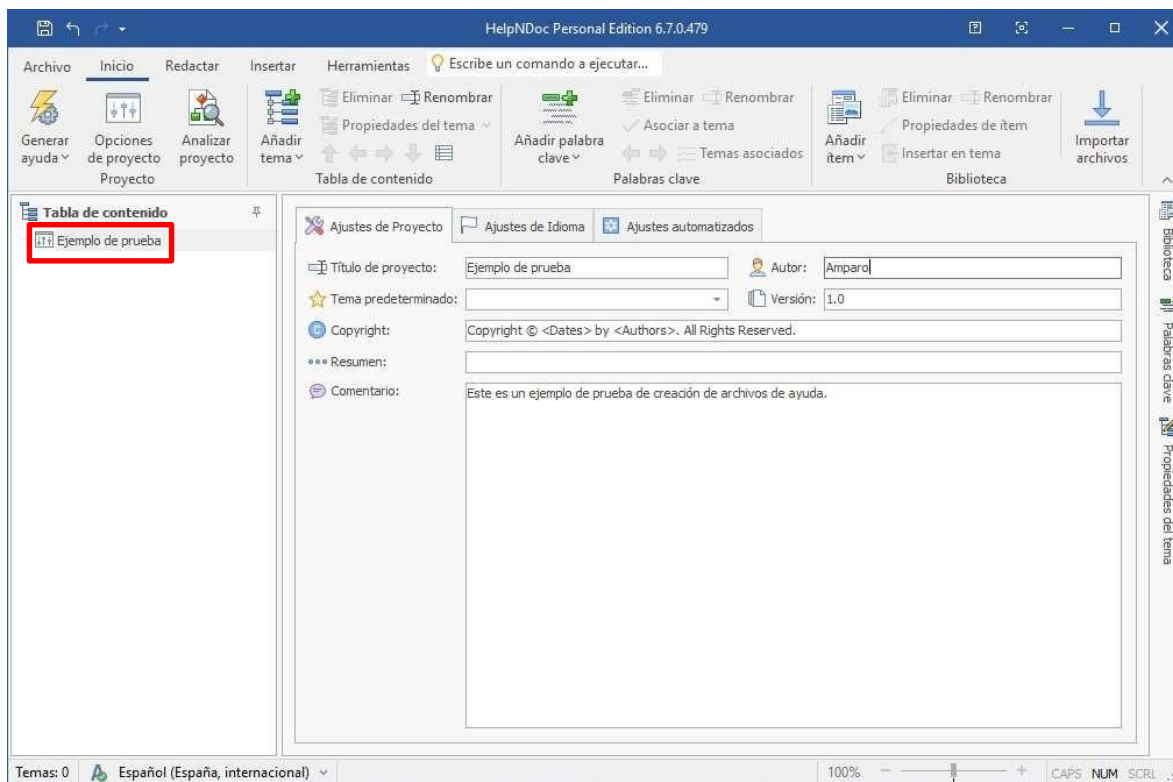
4.1. Generar documentación con HelpNDoc.

Una vez que se tiene la aplicación descargada e instalada en el equipo procederemos a la creación de un nuevo proyecto. Hay que pulsar en Archivo -> Nuevo Proyecto y le podemos dar nombre e idioma (además de una tabla de contenido). En las siguientes imágenes se muestran capturas de pantalla de este proceso.

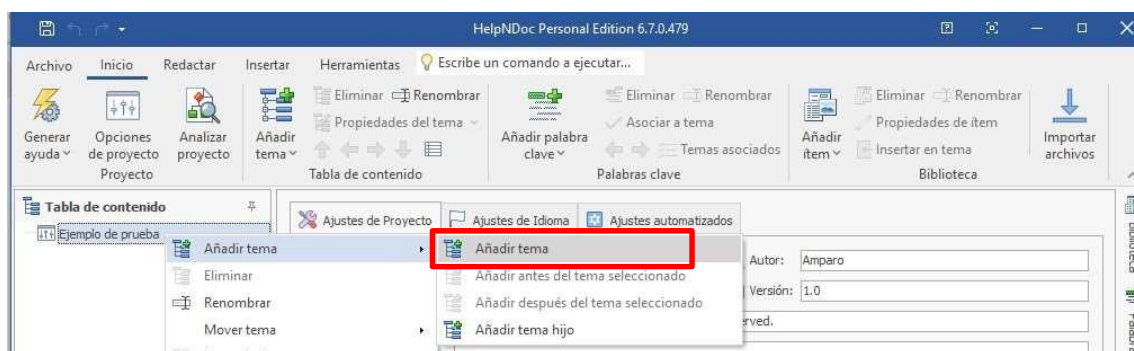


Una vez se han rellenado los campos, se pulsa en el botón Crear proyecto y aparecerá la ventana de Ajustes de Proyecto. En ella podemos seleccionar el nombre, autor, comentario del archivo de

ayuda, Copyright entre otros. Tras completar esto, ya estamos listos para empezar a crear el archivo de ayuda.

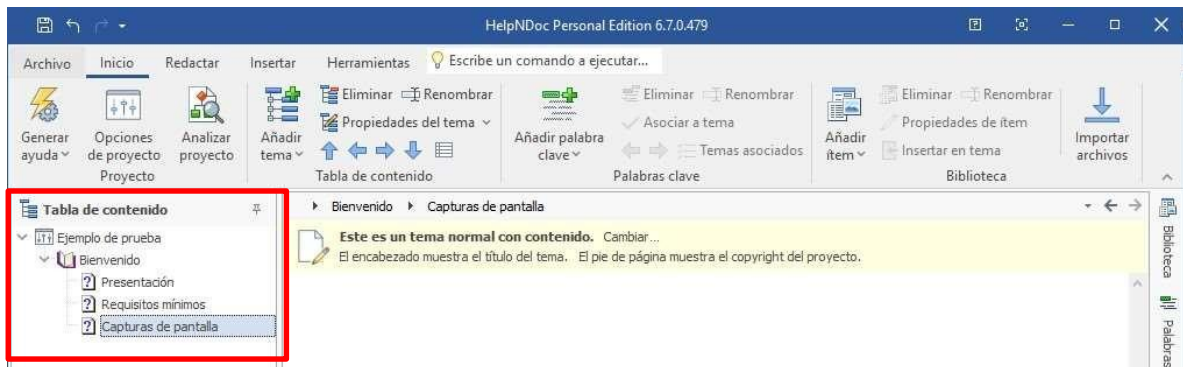


Lo primero que se va a explicar es cómo organizar estos archivos. En la imagen anterior se ve que en la parte de la izquierda se encuentra la Tabla de contenido y la raíz se llama Ejemplo de prueba. A partir de ella se pueden ir añadiendo diferentes nodos y así crear una jerarquía. En el ejemplo que estamos realizando se va a incluir unos cuantos nodos (todos ellos dependerán del nodo Ejemplo de prueba). Para ello, basta con hacer clic con el botón derecho encima de la raíz Ejemplo de prueba y pulsamos en la opción Añadir Tema -> Añadir tema tal y como se muestra en la imagen. También podemos seleccionar Añadir tema hijo y obtendremos el mismo resultado.



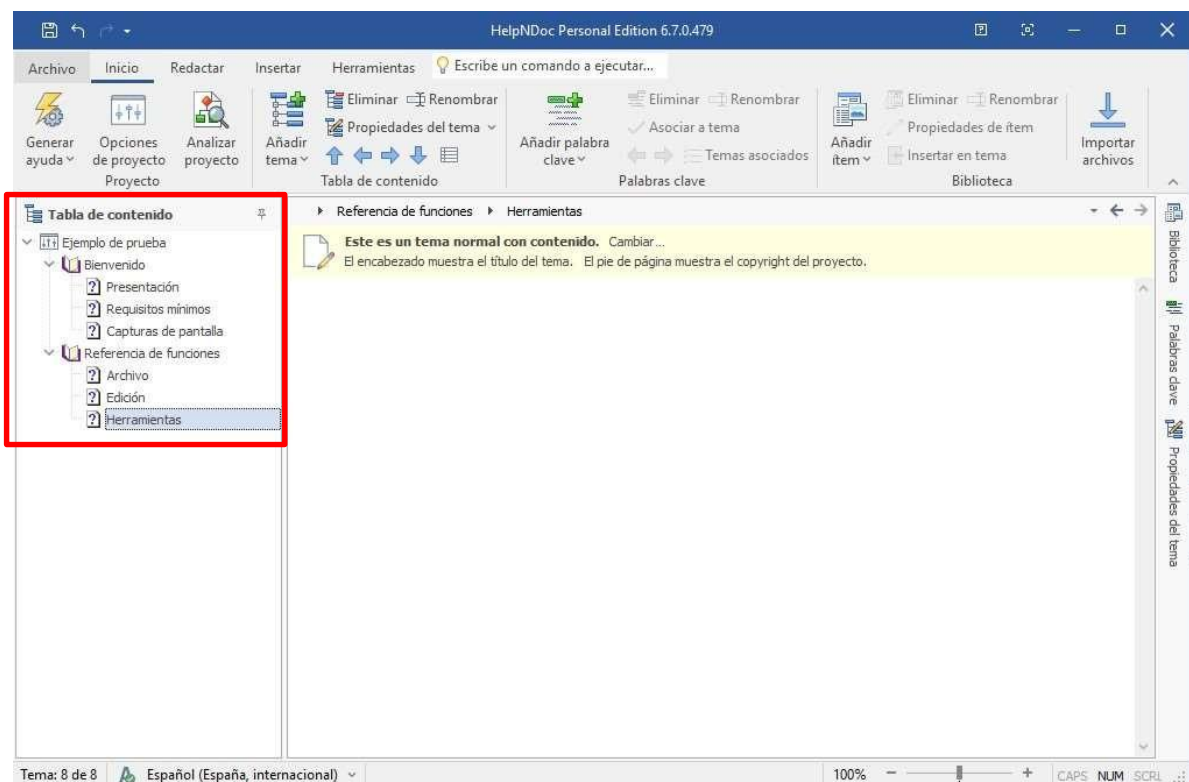
A este nuevo tema que hemos añadido vamos a darle nombre, por ejemplo, le llamaremos Bienvenido. Ahora vamos a añadir nodos hijo a este nuevo tema que hemos creado, para ello, tal y como se hizo antes, se hace clic con el botón derecho encima de Bienvenido y se selecciona Añadir tema hijo. Este proceso se va a realizar 3 veces para añadir tres nodos hijo que colgarán de

Bienvenido. A estos tres nodos se les llamará, Presentación, Requisitos mínimos y Capturas de pantalla.

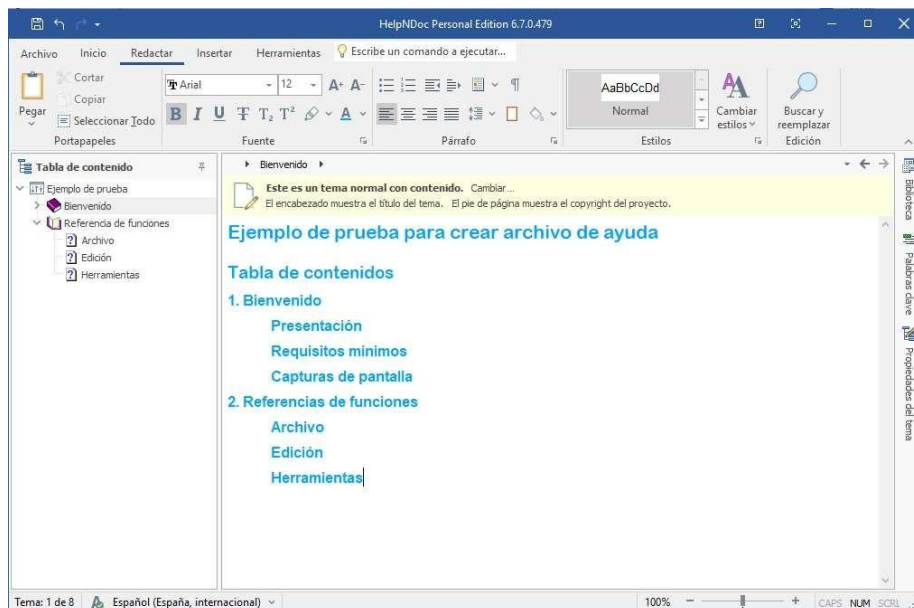


Como puede ver en la imagen anterior, el nodo Bienvenido ahora es como una especie de libro que si se despliega muestra los otros tres nodos que habíamos agregado. Ahora vamos a crear otro nodo que dependa directamente del nodo padre (Ejemplo de prueba) y le llamaremos

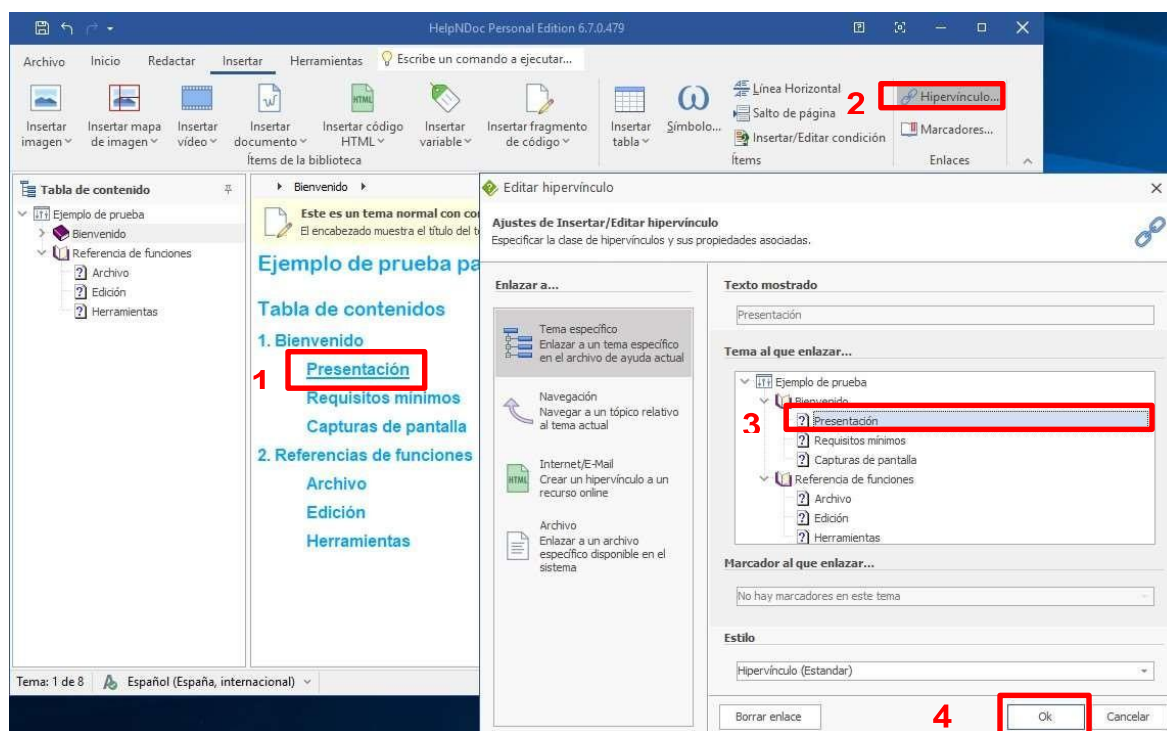
Referencia de funciones e incluiremos otros tres nodos hijos que dependan directamente de Referencia de funciones. En la siguiente captura se ve el resultado.



La estructura del archivo de ayuda ya está creada. Ahora sólo falta rellenar los diferentes campos. Vamos a situarnos en el nodo Bienvenido y en la zona de la derecha se puede empezar a escribir texto. En este caso se incluirá simplemente el índice para referenciar a las diferentes secciones del archivo de ayuda.



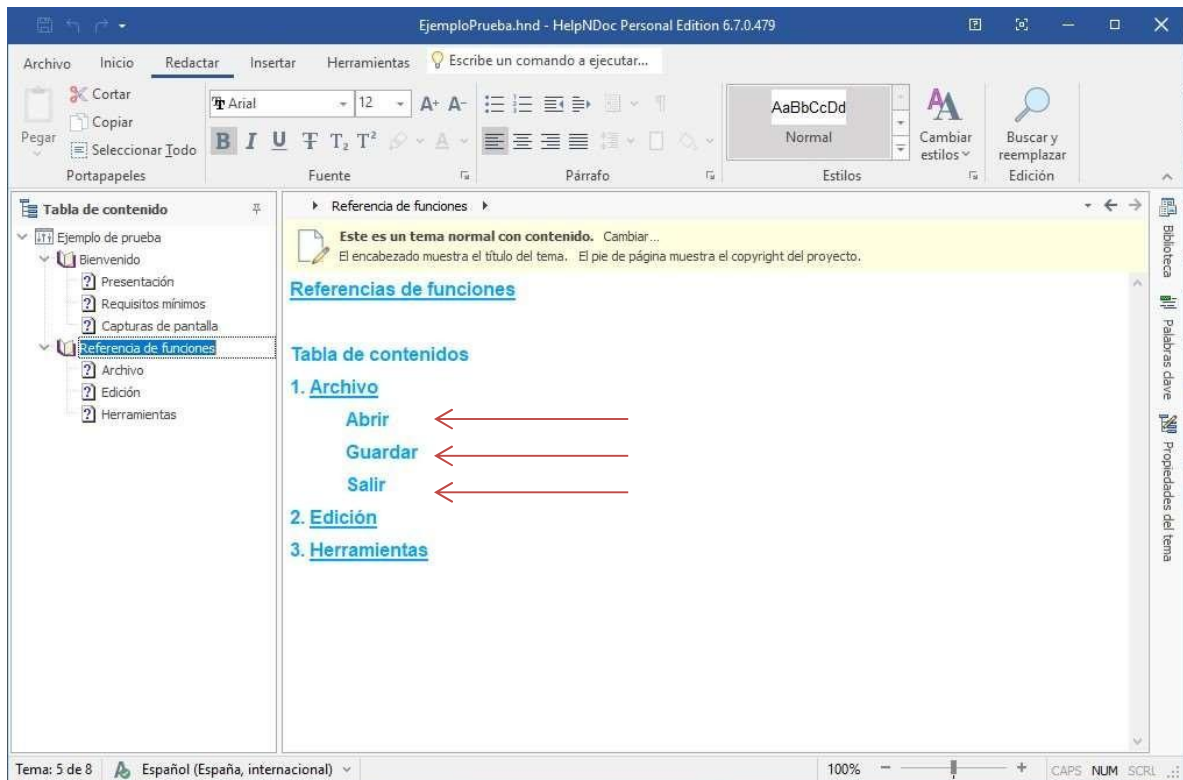
Ahora que se tiene la tabla de contenido se van a agregar las referencias para que al pulsar encima de alguna de las opciones, nos lleve directamente a la sección adecuada. Para ello se selecciona el texto que quiere referenciarse (por ejemplo, Presentación) y en la parte superior en la pestaña **Insertar**, se selecciona la opción Hipervínculo. En ese momento se va a mostrar un cuadro de diálogo para poder seleccionar el nodo al cual queremos referenciar el texto.



Ahora el texto Presentación ya está enlazado con el nodo correspondiente y cuando el proyecto esté finalizado, al hacer clic (dentro de la tabla de contenido) en la opción Presentación nos llevará directamente al nodo. Ahora, este proceso lo vamos a hacer con todos los nodos y agregaremos la referencia a todos los nodos.

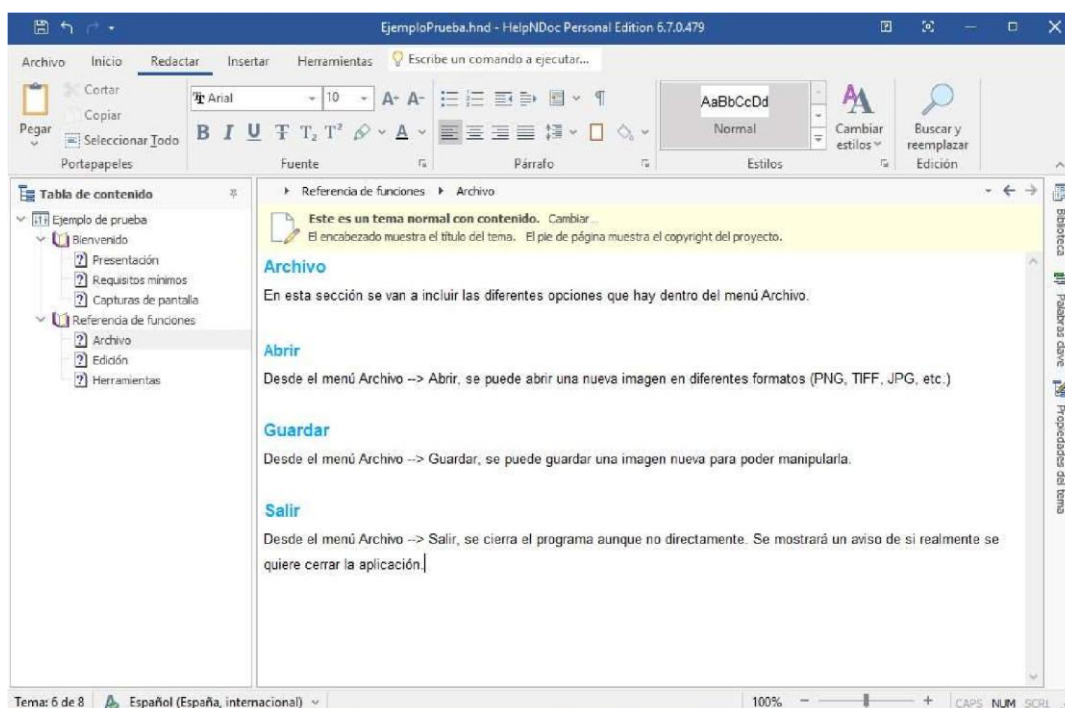
Una vez realizado esto, el siguiente paso es ir rellenando todos los nodos, por ejemplo, en el campo presentación se puede incluir una pequeña presentación de la aplicación.

Ahora vamos a ver algo relacionado con agregar referencias. Como hemos visto, simplemente hemos agregado referencias a un nodo en concreto, pero también se puede agregar referencias a partes del texto. Primeramente, vamos a crear la tabla de contenido (dentro del nodo Referencia de funciones), y vamos a incluir varias categorías dentro del nodo Archivo, tal y como se muestra en la imagen.

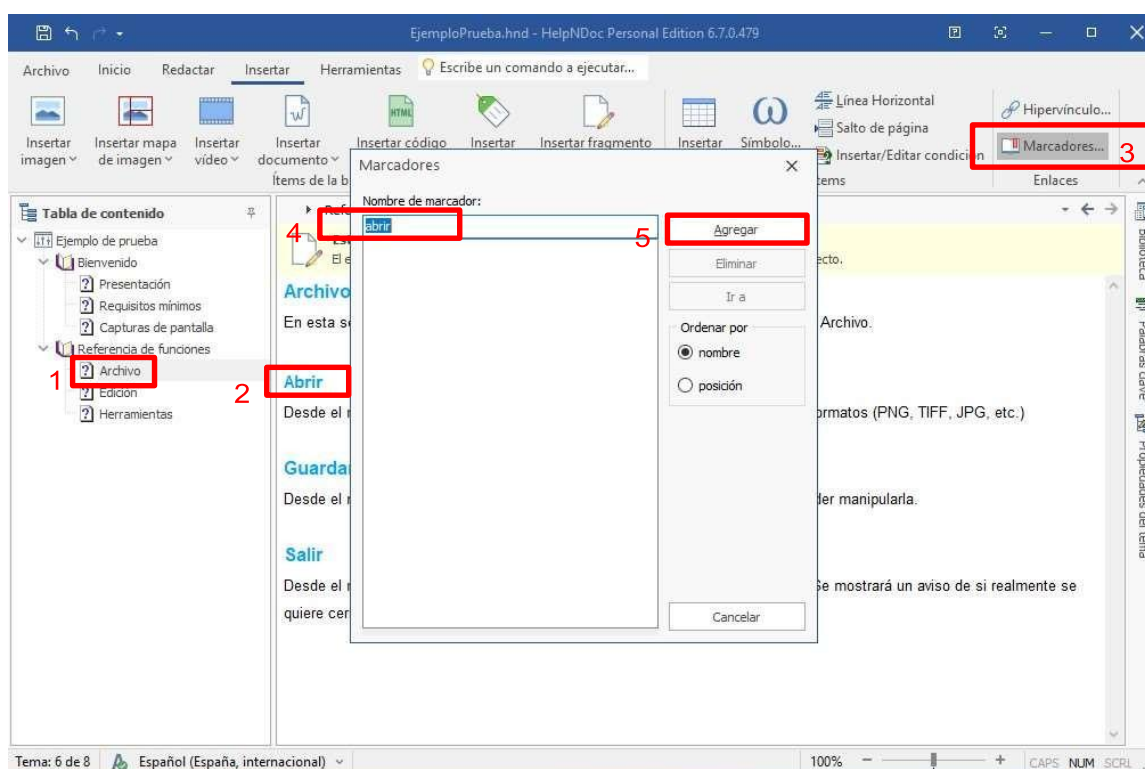


Es decir, hay varias categorías dentro del nodo Archivo, puesto que este campo tendrá diferentes partes. El siguiente paso es crear el texto asociado al nodo Archivo y como se ha dicho, este nodo va a tener tres partes bien diferenciadas.

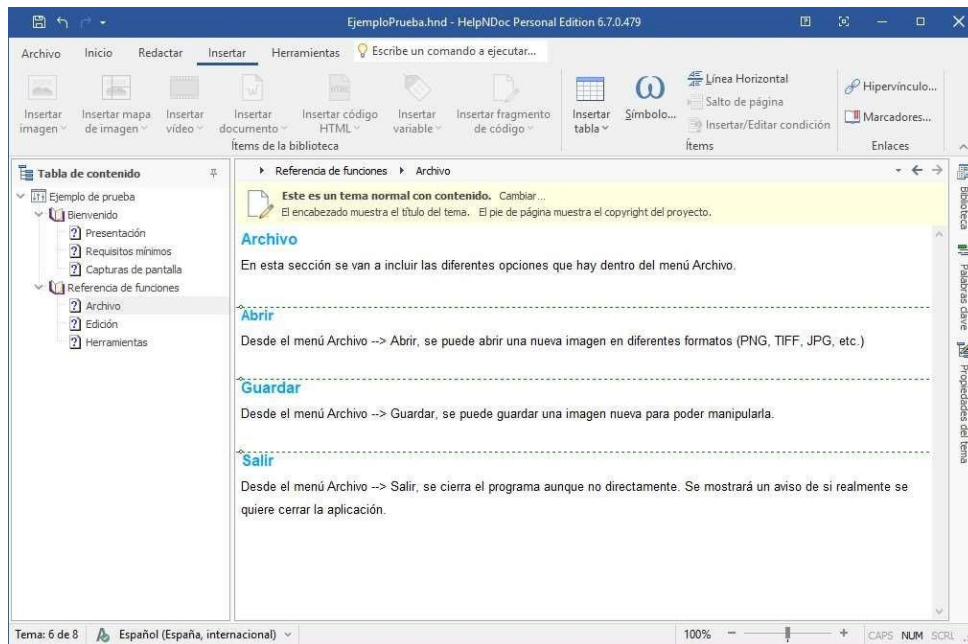
En la siguiente imagen se observa el texto que contendrá Archivo.



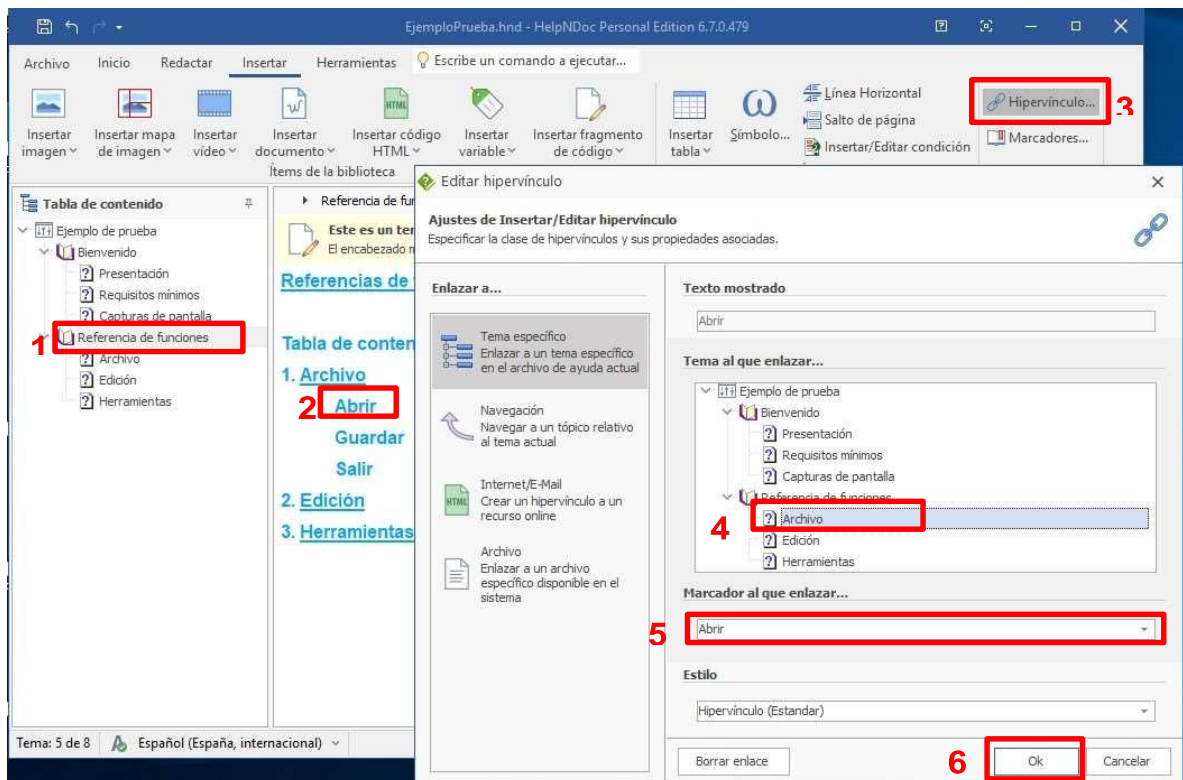
El texto que se ha incluido no es relevante ya que sólo estamos haciendo un pequeño ejemplo. El siguiente paso es, dentro de la tabla de contenido del nodo Referencia de funciones, agregar la referencia a la sección Abrir, Guardar y Salir. Para ello hay que hacer algo previamente, y es crear un ancla en las diferentes categorías para que pueda referenciarse a ellas. Seleccionamos el texto Abrir y vamos a la pestaña Insert y pulsamos en la opción que pone Insert/Remove anchor and aparecerá un cuadro de diálogo para dar nombre a esa ancla que hemos creado. En este caso le vamos a llamar abrir.



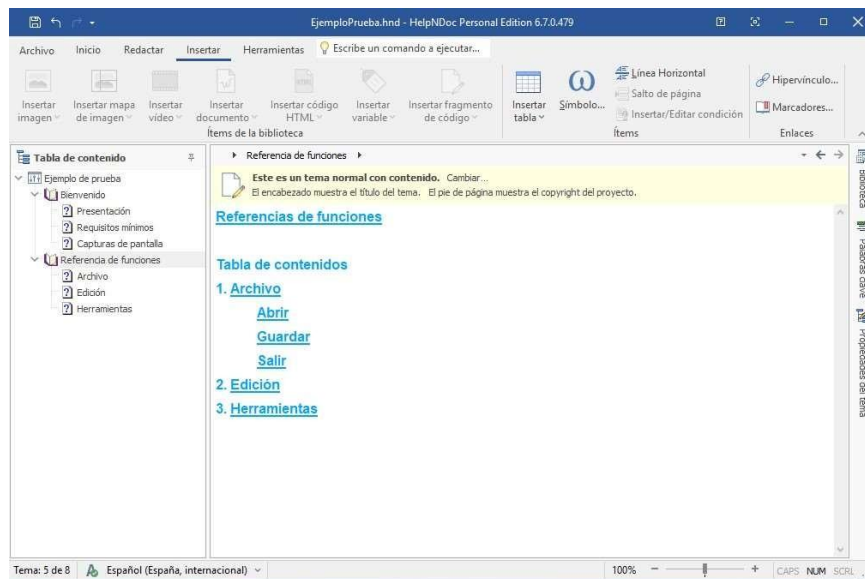
Este proceso se debe realizar con los textos Guardar y Salir, para agregar sus respectivos marcadores. Finalmente, el texto queda con este aspecto.



Ahora ya estamos preparados para hacer la referencia desde la tabla de contenido del nodo Referencia de funciones. El proceso es muy similar al seguido hasta ahora, se selecciona el texto (Abrir) y se pulsa en la pestaña Insertar y se hace clic en Hipervínculo. Ahora, dentro de la sección archivo, en la parte inferior se muestra un desplegable en el que poder seleccionar a qué marcador queremos hacer referencia, en este caso al marcador Abrir.



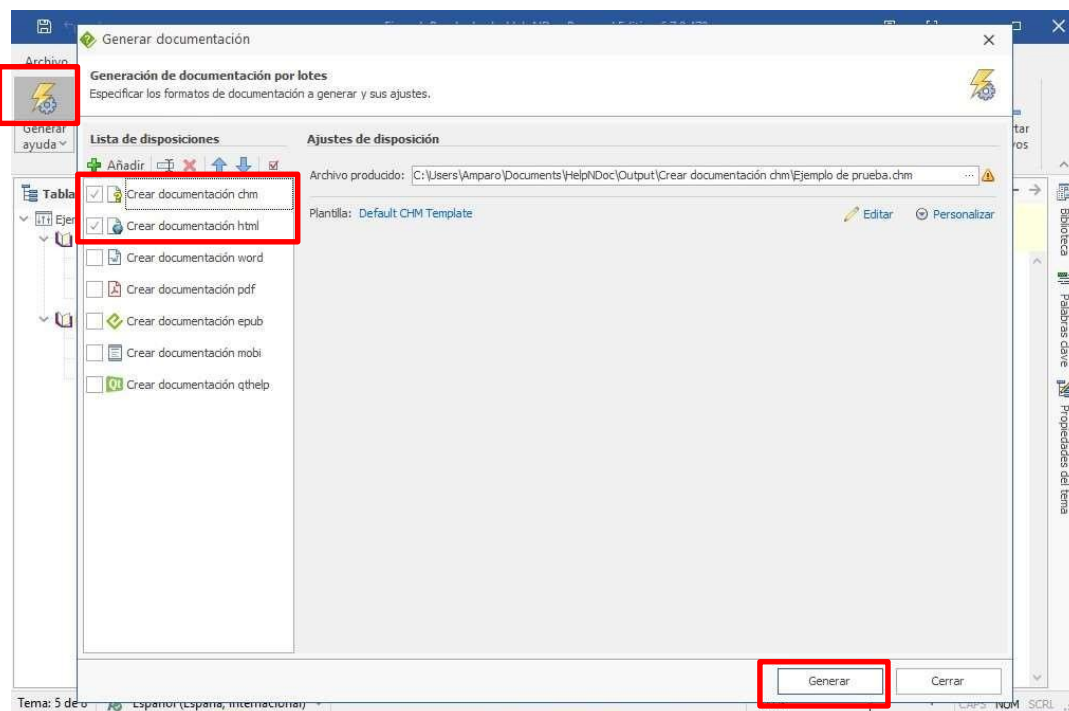
El proceso habría que realizarlo con toda la tabla de contenido dándonos este resultado:



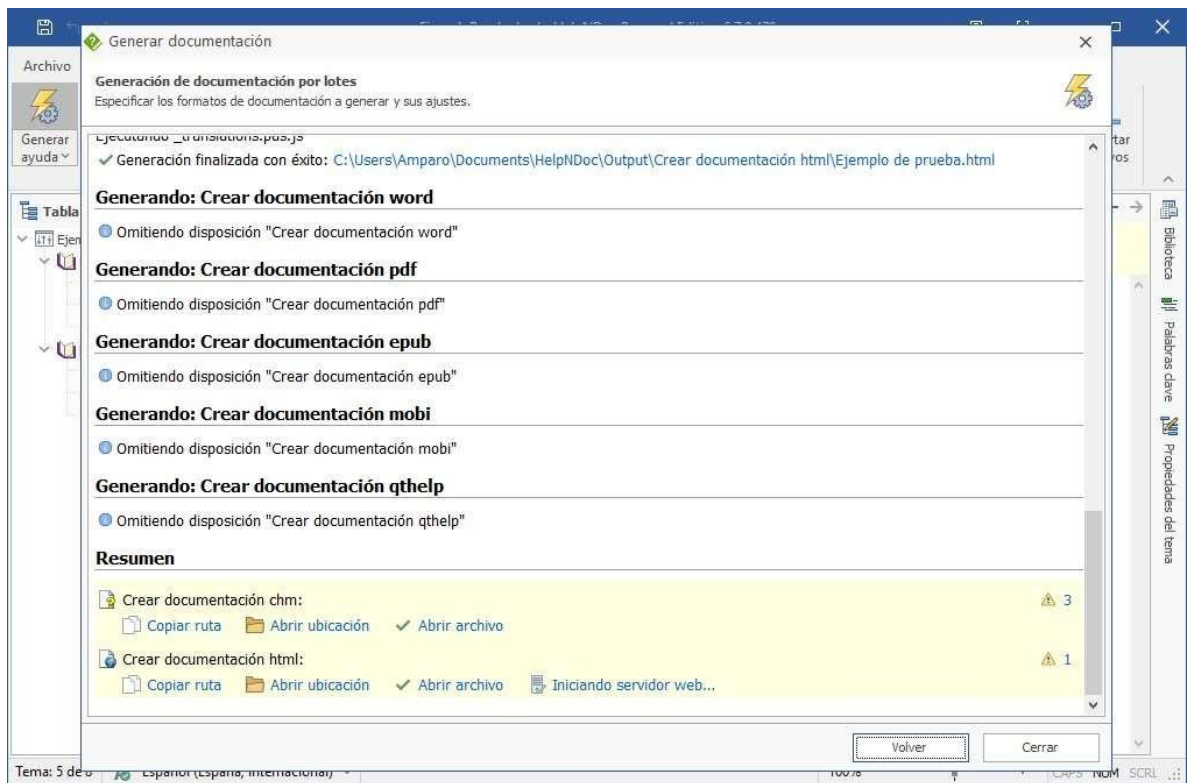
Y llegados a este punto, más o menos deberíamos tener una ligera idea de cómo funciona HelpNDoc.

Además de lo explicado, tiene muchas funciones típicas de un editor de texto, como insertar vídeos, tablas, imágenes, ecuaciones, palabras clave, etc.

Ahora queda el último paso que es generar el archivo de ayuda. Para ello basta con pulsar en la pestaña Inicio, hacer clic en Generar ayuda y se mostrará un cuadro de diálogo en el que poder seleccionar el tipo de archivo de ayuda que queremos (chm, pdf, html, entre otros). En nuestro ejemplo vamos a seleccionar el formato chm y html y tras ello, pulsamos en Generar.

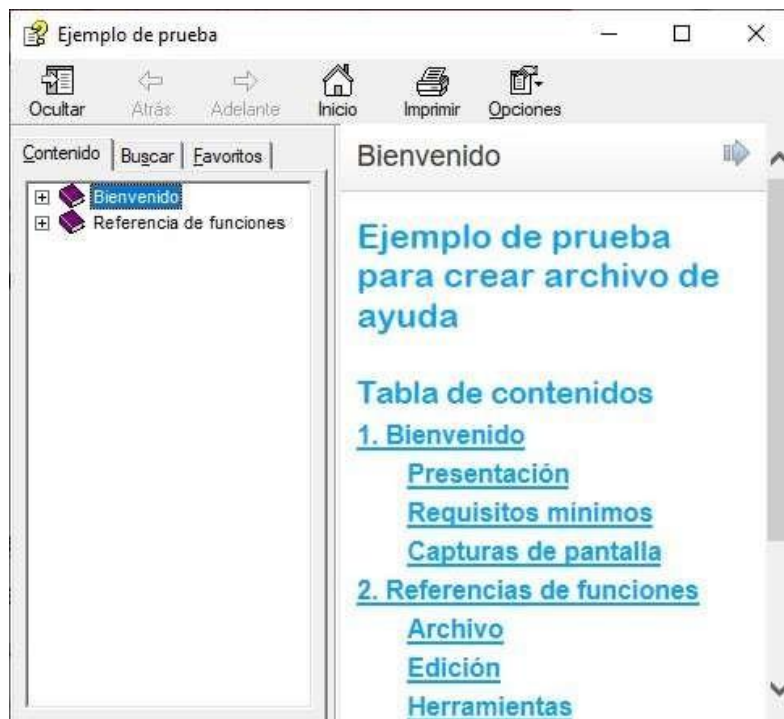


Así se genera la documentación mostrándonos la siguiente ventana. En el apartado Resumen nos indica la ruta en la cual se ha guardado la documentación.

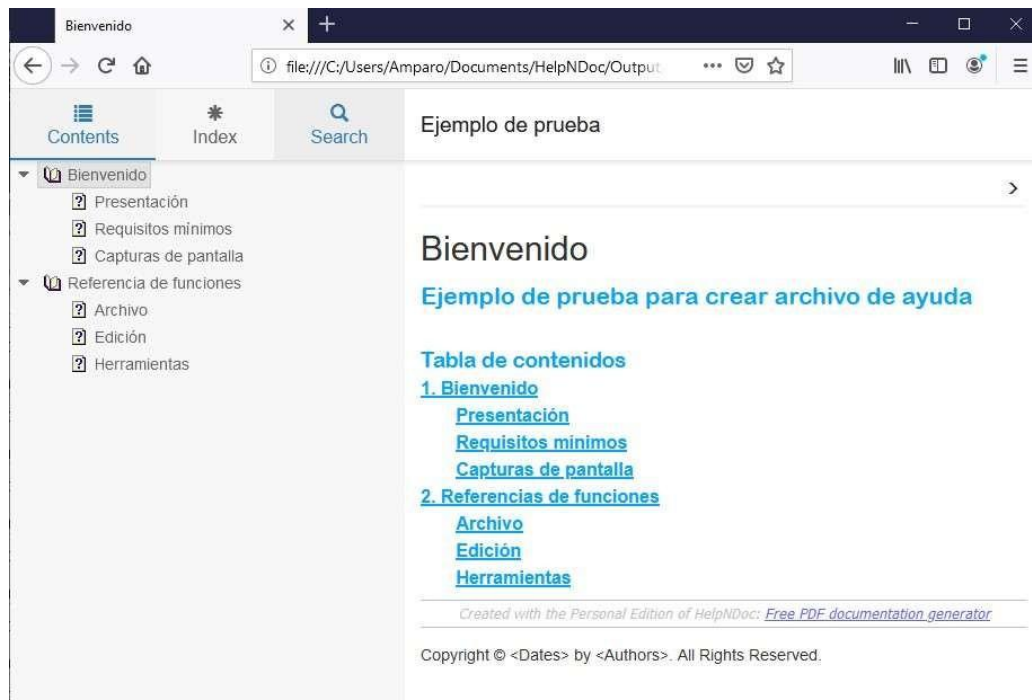


Tras unos segundos, se habrá generado el proyecto y vamos a ver el resultado.

Si vamos a la ruta del documento .chm y lo abrimos tenemos la siguiente documentación.



En cuanto a la documentación .html tenemos:



Como vemos los resultados son buenos y además HelpNDoc es muy sencillo de utilizar. Lo único, que al ser la versión de prueba como se observa en la imagen anterior, hay una marca de agua indicando que se está utilizando una versión de prueba.

4.2. Abrir documentación desde Visual Studio.

Por último, y ya para finalizar, vamos a ver cómo abrir el archivo .chm y .HTML desde Visual Studio. Abrimos un nuevo proyecto e incluimos en el formulario dos botones, uno Abrir documentación chm y Abrir documentación HTML tal y como se muestra en la imagen.

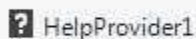


Antes que nada, vamos a guardar los archivos generados en la carpeta donde está el ejecutable de nuestra aplicación. "bin\Debug\Projectname.exe". Una vez guardado, vamos a escribir el siguiente código asociado a los botones:

```
private void btnHtml_Click(object sender, EventArgs e)
{
    string rutaEjecutable = System.IO.Directory.GetCurrentDirectory();
    System.Diagnostics.Process.Start(rutaEjecutable + "\\html\\Prueba.html");
}

private void btnChm_Click(object sender, EventArgs e)
{
    string rutaEjecutable = System.IO.Directory.GetCurrentDirectory();
    System.Diagnostics.Process.Start(rutaEjecutable + "\\chm\\Prueba.chm");
}
```

5. Incorporación de ayuda en Visual Studio.



HelpProvider1

Mediante el uso del componente HelpProvider, se pueden asociar temas de Ayuda de un archivo de Ayuda (en formato HTML o HTMLHelp 1.x o

superior) a controles específicos en formularios Windows Forms.

Para conectar un fichero de ayuda a un formulario arrastramos un control HelpProvider sobre el formulario y establecemos su propiedad HelpNamespace con el valor de la ruta completa al fichero .chm. Si el fichero .chm está en la misma carpeta que el fichero .exe de tu aplicación, puedes especificar únicamente el nombre del fichero (por ejemplo helpfile.chm).

Al añadir el control HelpProvider al formulario hará que haya propiedades nuevas disponibles para cada control del formulario. Estas propiedades son:

1. HelpKeyword.
2. HelpNavigator.
3. HelpString.
4. ShowHelp.

El comportamiento de la ayuda contextual variará dependiendo de los valores asignados a estas propiedades.

En primer lugar, si se asigna el valor False a la propiedad ShowHelp, no se mostrará ayuda contextual.

Asocia los temas de ayuda con los controles como sigue:

1. Establece la propiedad ShowHelp del control a True.
2. Establece la propiedad HelpKeyword en la ventana Propiedades. Esta es la cadena que se pasa al archivo de Ayuda a través del componente HelpProvider para abrir el tema de Ayuda apropiado.
3. En la ventana Propiedades, establece la propiedad HelpNavigator en un valor de la enumeración HelpNavigator.

Esto determina de qué forma se pasa la propiedad HelpKeyword al sistema de Ayuda. A continuación, se muestra una tabla de posibles configuraciones y sus descripciones.

Nombre de	Descripción miembro
AssociateIndex	Especifica que el índice de un tema concreto se ejecuta en la dirección URL determinada.
Find	Especifica que se muestra la página de búsqueda de una dirección URL determinada.
Index	Especifica que se muestra el índice de una dirección URL determinada.
KeywordIndex	Especifica una palabra clave para buscar y la acción que debe ejecutarse en la dirección URL determinada.
TableOfContents	Especifica que se muestra la tabla de contenido del archivo de Ayuda HTML 1.0.
Topic	Especifica que se muestra el tema al que hace referencia la dirección URL determinada.

4. Asegúrate de que la propiedad HelpString está en blanco.

En tiempo de ejecución, si se presiona F1 cuando el control (del que se han establecido las propiedades HelpKeyword y HelpNavigator) tiene el foco, se abrirá el archivo de Ayuda asociado con ese componente HelpProvider.

Referencias:

- <http://www.helpscribble.com/dotnet.html>
- <https://msdn.microsoft.com/es-es/library/s3ztx0at%28v=vs.110%29.aspx>

5.1. Ejemplo de uso del control HelpProvider.

En primer lugar, si únicamente deseamos que en tiempo de ejecución de nuestra aplicación y al pulsar F1 se muestre el fichero de ayuda .chm deberemos incorporar a nuestro formulario un control HelpProvider y establecer su propiedad HelpNamespace con el valor de la ruta completa al fichero .chm. Recuerda que si el fichero .chm está en la misma carpeta que el fichero .exe de tu aplicación, puedes especificar únicamente el nombre del fichero. Para que funcione, deberás además poner la propiedad del formulario ShowHelp a true.

Por otro lado, el control HelpProvider nos permite mostrar mensajes de ayuda al usuario en los controles que tenemos en nuestra interfaz para que el usuario sepa cómo utilizarlos o pueda obtener una descripción sobre la función que tiene el control seleccionado.

En el siguiente ejemplo veremos cómo configurar el formulario para poder utilizar este control y cómo colocar mensajes de ayuda en los controles de la interfaz de usuario del formulario.

Crearemos un nuevo proyecto llamado UsoHelpProvider. El diseño del formulario será el siguiente:



A continuación, configuramos el formulario para que en la barra de herramientas aparezca el símbolo con el signo de: **?** que servirá para que el usuario solicite la ayuda para algún control. Para ello en las propiedades del formulario buscamos la propiedad `HelpButton` y la ponemos a `True`.

Con esto todavía no cambia nada en el formulario porque hay que eliminar el botón de maximizar y el de minimizar. Para ello ponemos las propiedades `MaximizeBox` y `MinimizeBox` a `False`.

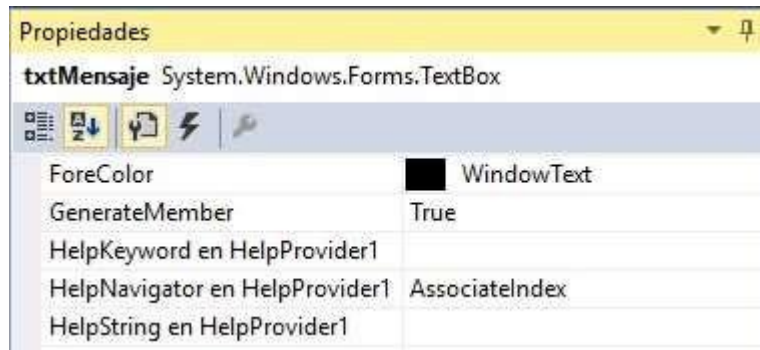
Tras esto el formulario queda con este aspecto, apareciendo el botón de ayuda.



Si ejecutamos la aplicación y hacemos clic sobre el botón de ayuda y luego sobre un control del formulario, todavía no nos muestra ninguna ayuda. Esto es porque falta el control `HelpProvider`.

Así que incorporamos a nuestro formulario el control `HelpProvider`, haciendo doble clic sobre el mismo en el cuadro de herramientas.

Ahora podemos observar que en los controles de nuestro formulario han aparecido nuevas propiedades. Sirvan de ejemplo las que aparecen en el `TextBox`.



Vamos ahora a colocar un mensaje de ayuda en el TextBox. Para ello escribimos “Escribe aquí el mensaje que deseas mostrar” en la propiedad HelpString en HelpProvider1 de txtMensaje.

Seguidamente hacemos lo mismo con el botón poniendo en la propiedad HelpString en HelpProvider1 “Este botón permite mostrar el mensaje”.

Ejecutamos la aplicación y comprobamos que efectivamente ahora aparecen los mensajes de ayuda al pulsar sobre los controles siempre y cuando hayamos pulsado sobre el botón de ayuda primero.

Completamos ahora el código del evento Click del botón.

```
private void btnMostrar_Click(object sender, EventArgs e)
{
    lblMensaje.Text = txtMensaje.Text;
}
```

Si modificamos la propiedad ShowHelp del botón a False ya no se muestra la ayuda de ese control. Hacemos la modificación y lo comprobamos.

Si tenemos un fichero de ayuda para la aplicación con extensión .chm como el que elaboramos en el apartado 4 podemos utilizarlo poniendo en la propiedad HelpNamespace del control HelpProvider el nombre del fichero siempre y cuando el fichero .chm esté en la misma carpeta que el fichero .exe de tu aplicación. En caso contrario, habrá que poner la ruta completa del archivo.

Esta ayuda se muestra al arrancar la aplicación y pulsar F1.

Si queremos que al pulsar los controles del formulario aparezca el fichero de ayuda en lugar de los textos de ayuda anteriores debemos dejar la propiedad HelpString de cada control en blanco.

Además, podemos determinar en qué modo se abrirá la ayuda mediante la propiedad HelpNavigator que está descrita anteriormente.

5. 2. Ayuda contextual.

La ayuda contextual permite al sistema brindar soporte al usuario de forma rápida y directa indicándole mediante elementos visuales o textuales una descripción sobre la acción que debe desarrollar o la funcionalidad de un elemento determinado.

5.2.1. ToolTips

El uso de elementos textuales que aparecen en la interfaz al posicionarse el usuario con puntero del ratón durante un par de segundos sobre un elemento se conoce como Tooltip. Este tipo de

elementos permiten dotar a la aplicación de una ayuda contextual cuyo objetivo es facilitar al usuario el uso de la aplicación proporcionándole información de apoyo sobre la funcionalidad de un elemento.

5.2.1.1. Introducción al componente ToolTip de Windows Forms.

El componente ToolTip de los formularios de Windows muestra texto cuando el usuario apunta a controles. Un ToolTip puede ser asociado con cualquier control. Un ejemplo de uso de este componente sería que, para ahorrar espacio en un formulario, se mostrara un icono pequeño en un botón y usar un ToolTip para explicar la función del botón.

Un componente ToolTip proporciona la propiedad ToolTip a múltiples controles en un formulario Windows u otro contenedor. Por ejemplo, si colocamos un componente ToolTip en un formulario, podemos mostrar “Escribe tu nombre aquí” para un control TextBox y “Haz clic aquí para guardar cambios” para un control Button.

Los métodos clave para el componente ToolTip son SetToolTip y GetToolTip. Podemos usar el método SetToolTip para establecer los ToolTips que muestran los controles.

Las propiedades clave son Active, que debe establecerse a true para que aparezca el ToolTip, and AutomaticDelay, que establece la duración de tiempo en que la cadena de ToolTip se muestra, cuánto tiempo debe apuntar el usuario al control para que el ToolTip aparezca y cuánto tiempo por consiguiente le cuesta a la ventana ToolTip aparecer.

5.2.1.2. Establecer ToolTips para controles en Formularios de Windows.

Deberemos seguir los siguientes pasos:

1. Añade un control ToolTip al formulario. Puedes establecer sus propiedades ToolTipTitle (determina el título de la información sobre las herramientas) y ToolTipIcon (determina el icono que se muestra en la información sobre las herramientas).
2. Selecciona el control que mostrará el ToolTip o añádelo al formulario.
3. En la ventana de propiedades del anterior control establece el valor de ToolTip on ToolTip1 a un texto apropiado.